

2.a

```
In [1]: import os
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
from xgboost import XGBClassifier
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Set random seed for reproducibility
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)

# Configuration
DATASET_PATH = '/content/drive/MyDrive/UCMerced_LandUse/Images'
BATCH_SIZE = 32
NUM_WORKERS = 4
PREFETCH_FACTOR = 2
PIN_MEMORY = True

# Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Define transforms using ImageNet statistics
transform = transforms.Compose([
    transforms.Resize((224, 224)), # ResNet expects 224x224
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

class UCMercedDataset(Dataset):

    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
```

```

        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]
        image = Image.open(img_path).convert('RGB')
        label = self.labels[idx]

        if self.transform:
            image = self.transform(image)

        return image, label

def load_dataset(dataset_path):

    # Get all class folders
    classes = sorted(os.listdir(dataset_path))
    classes = [c for c in classes if os.path.isdir(os.path.join(dataset_path, c))]

    # Create class to index mapping
    class_to_idx = {cls_name: idx for idx, cls_name in enumerate(classes)}
    idx_to_class = {idx: cls_name for cls_name, idx in class_to_idx.items()}

    print(f"Found {len(classes)} classes:")
    for cls_name, idx in class_to_idx.items():
        print(f" {idx}: {cls_name}")

    # Collect all image paths and labels
    image_paths = []
    labels = []

    for cls_name in classes:
        cls_path = os.path.join(dataset_path, cls_name)
        cls_idx = class_to_idx[cls_name]

        images = sorted([f for f in os.listdir(cls_path) if f.endswith((''.png', '.j

        for img_name in images:
            img_path = os.path.join(cls_path, img_name)
            image_paths.append(img_path)
            labels.append(cls_idx)

    return image_paths, labels, class_to_idx, idx_to_class

def create_balanced_splits(image_paths, labels, train_size=0.7, val_size=0.15, test

    assert abs(train_size + val_size + test_size - 1.0) < 1e-6, "Splits must sum to

    # First split: train vs (val + test)
    X_train, X_temp, y_train, y_temp = train_test_split(
        image_paths, labels,
        test_size=(val_size + test_size),
        stratify=labels,
        random_state=SEED

```

```

)

# Second split: val vs test
val_ratio = val_size / (val_size + test_size)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=(1 - val_ratio),
    stratify=y_temp,
    random_state=SEED
)

print(f"\nDataset splits:")
print(f"  Training: {len(X_train)} images")
print(f"  Validation: {len(X_val)} images")
print(f"  Test: {len(X_test)} images")

return X_train, X_val, X_test, y_train, y_val, y_test

def load_pretrained_resnet(model_name='resnet50'):

    if model_name == 'resnet18':
        model = models.resnet18(pretrained=True)
        feature_dim = 512
    elif model_name == 'resnet34':
        model = models.resnet34(pretrained=True)
        feature_dim = 512
    elif model_name == 'resnet50':
        model = models.resnet50(pretrained=True)
        feature_dim = 2048
    elif model_name == 'resnet101':
        model = models.resnet101(pretrained=True)
        feature_dim = 2048
    elif model_name == 'resnet152':
        model = models.resnet152(pretrained=True)
        feature_dim = 2048
    else:
        raise ValueError(f"Unsupported model: {model_name}")

    # Remove the final fully connected layer (classification head)

    model = nn.Sequential(*list(model.children())[:-1])

    # Set to evaluation mode and freeze parameters
    model.eval()
    for param in model.parameters():
        param.requires_grad = False

    # Move model to GPU
    model = model.to(device)
    print(f"\nLoaded {model_name} as feature extractor")
    print(f"Feature dimension: {feature_dim}")
    print(f"Model on device: {next(model.parameters()).device}")

    return model, feature_dim

```

```

def extract_features(model, dataloader, feature_dim):

    features_list = []
    labels_list = []

    print(f"Expected feature shape per image: {feature_dim}")

    with torch.no_grad(): # No gradients needed for feature extraction
        for batch_idx, (images, labels) in enumerate(tqdm(dataloader, desc="Extract
            # Send images to GPU
            images = images.to(device)

            # Extract features - output shape is (B, feature_dim, 1, 1)
            features = model(images)

            # Reshape to (B, feature_dim) by removing spatial dimensions
            features = features.squeeze()

            if features.dim() == 1:
                features = features.unsqueeze(0)

            # Move features back to CPU and convert to numpy
            features_cpu = features.cpu().numpy()
            labels_cpu = labels.numpy()

            features_list.append(features_cpu)
            labels_list.append(labels_cpu)

            # Print batch info for first batch
            if batch_idx == 0:
                print(f" Batch shape (GPU): {images.shape}")
                print(f" Features shape (CPU): {features_cpu.shape}")

        # Concatenate all batches
        features = np.vstack(features_list)
        labels = np.concatenate(labels_list)

        print(f"\nFinal extracted features shape: {features.shape}")
        print(f"Final labels shape: {labels.shape}")

        return features, labels

# Load dataset
print("Loading dataset...")
image_paths, labels, class_to_idx, idx_to_class = load_dataset(DATASET_PATH)

# Create splits
X_train, X_val, X_test, y_train, y_val, y_test = create_balanced_splits(
    image_paths, labels,
    train_size=0.7,
    val_size=0.15,
    test_size=0.15
)

```

```

# Create datasets
train_dataset = UCMercedDataset(X_train, y_train, transform=transform)
val_dataset = UCMercedDataset(X_val, y_val, transform=transform)
test_dataset = UCMercedDataset(X_test, y_test, transform=transform)

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    prefetch_factor=PREFETCH_FACTOR,
    pin_memory=PIN_MEMORY
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    prefetch_factor=PREFETCH_FACTOR,
    pin_memory=PIN_MEMORY
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    prefetch_factor=PREFETCH_FACTOR,
    pin_memory=PIN_MEMORY
)

print(f"\nDataLoader Configuration:")
print(f"  Batch size: {BATCH_SIZE}")
print(f"  Num workers: {NUM_WORKERS}")
print(f"  Prefetch factor: {PREFETCH_FACTOR}")
print(f"  Pin memory: {PIN_MEMORY}")

# Load pretrained model (returns model and feature dimension)
resnet_model, feature_dim = load_pretrained_resnet('resnet50')

# STEP 2: FEATURE EXTRACTION

# Check if features are already saved
FEATURES_DIR = '/content/drive/MyDrive/ucmerced_features'
os.makedirs(FEATURES_DIR, exist_ok=True)

train_features_path = os.path.join(FEATURES_DIR, 'train_features.npy')
train_labels_path = os.path.join(FEATURES_DIR, 'train_labels.npy')
val_features_path = os.path.join(FEATURES_DIR, 'val_features.npy')
val_labels_path = os.path.join(FEATURES_DIR, 'val_labels.npy')
test_features_path = os.path.join(FEATURES_DIR, 'test_features.npy')
test_labels_path = os.path.join(FEATURES_DIR, 'test_labels.npy')

```

```

if os.path.exists(train_features_path):
    print("\n" + "="*50)
    print("Loading pre-extracted features from disk...")
    print("="*50)
    train_features = np.load(train_features_path)
    train_labels = np.load(train_labels_path)
    val_features = np.load(val_features_path)
    val_labels = np.load(val_labels_path)
    test_features = np.load(test_features_path)
    test_labels = np.load(test_labels_path)
    print(f"Loaded train features: {train_features.shape}")
    print(f"Loaded val features: {val_features.shape}")
    print(f"Loaded test features: {test_features.shape}")
else:
    print("\n" + "="*50)
    print("EXTRACTING FEATURES (This will take a few minutes...)")
    print("="*50)

    # Extract features for training set
    print("\nExtracting training features...")
    train_features, train_labels = extract_features(resnet_model, train_loader, fea

    # Extract features for validation set
    print("\nExtracting validation features...")
    val_features, val_labels = extract_features(resnet_model, val_loader, feature_d

    # Extract features for test set
    print("\nExtracting test features...")
    test_features, test_labels = extract_features(resnet_model, test_loader, featur

    # Save features to disk for reuse
    print("\nSaving features to disk for future reuse...")
    np.save(train_features_path, train_features)
    np.save(train_labels_path, train_labels)
    np.save(val_features_path, val_features)
    np.save(val_labels_path, val_labels)
    np.save(test_features_path, test_features)
    np.save(test_labels_path, test_labels)
    print(f"Features saved to: {FEATURES_DIR}")

print("\n" + "="*50)
print("FEATURE EXTRACTION COMPLETE")
print("="*50)
print(f"Train features shape: {train_features.shape} (B x N format)")
print(f"Val features shape: {val_features.shape}")
print(f"Test features shape: {test_features.shape}")
print(f"Feature dimension (N): {train_features.shape[1]}")

# Train SVM classifier
print("\n" + "="*50)
print("Training SVM classifier...")
print("="*50)
svm_classifier = SVC(kernel='rbf', C=10, gamma='scale', random_state=SEED)
svm_classifier.fit(train_features, train_labels)

```

```

# Evaluate on validation set
svm_val_pred = svm_classifier.predict(val_features)
svm_val_acc = accuracy_score(val_labels, svm_val_pred)
print(f"\nSVM Validation Accuracy: {svm_val_acc:.4f}")

# Evaluate on test set
svm_test_pred = svm_classifier.predict(test_features)
svm_test_acc = accuracy_score(test_labels, svm_test_pred)
print(f"SVM Test Accuracy: {svm_test_acc:.4f}")

print("\nSVM Classification Report (Test Set):")
print(classification_report(test_labels, svm_test_pred, target_names=[idx_to_class[

# Train XGBoost classifier
print("\n" + "="*50)
print("Training XGBoost classifier...")
print("="*50)
xgb_classifier = XGBClassifier(
    n_estimators=200,
    max_depth=6,
    learning_rate=0.1,
    random_state=SEED,
    eval_metric='mlogloss'
)
xgb_classifier.fit(train_features, train_labels)

# Evaluate on validation set
xgb_val_pred = xgb_classifier.predict(val_features)
xgb_val_acc = accuracy_score(val_labels, xgb_val_pred)
print(f"\nXGBoost Validation Accuracy: {xgb_val_acc:.4f}")

# Evaluate on test set
xgb_test_pred = xgb_classifier.predict(test_features)
xgb_test_acc = accuracy_score(test_labels, xgb_test_pred)
print(f"XGBoost Test Accuracy: {xgb_test_acc:.4f}")

print("\nXGBoost Classification Report (Test Set):")
print(classification_report(test_labels, xgb_test_pred, target_names=[idx_to_class[

# Plot confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(20, 8))

# SVM Confusion Matrix
svm_cm = confusion_matrix(test_labels, svm_test_pred)
sns.heatmap(svm_cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
            xticklabels=[idx_to_class[i] for i in range(len(class_to_idx))],
            yticklabels=[idx_to_class[i] for i in range(len(class_to_idx))])
axes[0].set_title(f'SVM Confusion Matrix (Acc: {svm_test_acc:.4f})')
axes[0].set_ylabel('True Label')
axes[0].set_xlabel('Predicted Label')
plt.setp(axes[0].get_xticklabels(), rotation=45, ha='right')
plt.setp(axes[0].get_yticklabels(), rotation=0)

# XGBoost Confusion Matrix
xgb_cm = confusion_matrix(test_labels, xgb_test_pred)
sns.heatmap(xgb_cm, annot=True, fmt='d', cmap='Greens', ax=axes[1],

```

```

        xticklabels=[idx_to_class[i] for i in range(len(class_to_idx))],
        yticklabels=[idx_to_class[i] for i in range(len(class_to_idx))])
axes[1].set_title(f'XGBoost Confusion Matrix (Acc: {xgb_test_acc:.4f})')
axes[1].set_ylabel('True Label')
axes[1].set_xlabel('Predicted Label')
plt.setp(axes[1].get_xticklabels(), rotation=45, ha='right')
plt.setp(axes[1].get_yticklabels(), rotation=0)

plt.tight_layout()
plt.show()

# STEP 3: DETAILED PERFORMANCE ANALYSIS

print("\n" + "-"*70)
print("STEP 3: CLASSIFICATION RESULTS AND ANALYSIS")
print("-"*70)

# Get detailed metrics for both classifiers
from sklearn.metrics import precision_recall_fscore_support

# SVM Metrics
svm_precision, svm_recall, svm_f1, _ = precision_recall_fscore_support(
    test_labels, svm_test_pred, average='weighted', zero_division=0
)

# XGBoost Metrics
xgb_precision, xgb_recall, xgb_f1, _ = precision_recall_fscore_support(
    test_labels, xgb_test_pred, average='weighted', zero_division=0
)

# Print comprehensive metrics
print("\n" + "-"*70)
print("SVM CLASSIFIER PERFORMANCE")
print("-"*70)
print(f"Accuracy:  {svm_test_acc:.4f}")
print(f"Precision: {svm_precision:.4f}")
print(f"Recall:    {svm_recall:.4f}")
print(f"F1-Score:   {svm_f1:.4f}")

print("\n" + "-"*70)
print("XGBOOST CLASSIFIER PERFORMANCE")
print("-"*70)
print(f"Accuracy:  {xgb_test_acc:.4f}")
print(f"Precision: {xgb_precision:.4f}")
print(f"Recall:    {xgb_recall:.4f}")
print(f"F1-Score:   {xgb_f1:.4f}")

# Plot regular and normalized confusion matrices
fig, axes = plt.subplots(2, 2, figsize=(24, 20))

class_names = [idx_to_class[i] for i in range(len(class_to_idx))]

# SVM Confusion Matrix (Raw Counts)
svm_cm = confusion_matrix(test_labels, svm_test_pred)

```



```

sns.heatmap(svm_cm, annot=True, fmt='d', cmap='Blues', ax=axes[0, 0],
            xticklabels=class_names, yticklabels=class_names, cbar_kws={'label': 'C
axes[0, 0].set_title(f'SVM Confusion Matrix - Raw Counts\n(Test Accuracy: {svm_test
axes[0, 0].set_ylabel('True Label', fontsize=12)
axes[0, 0].set_xlabel('Predicted Label', fontsize=12)
plt.setp(axes[0, 0].get_xticklabels(), rotation=45, ha='right', fontsize=9)
plt.setp(axes[0, 0].get_yticklabels(), rotation=0, fontsize=9)

# SVM Normalized Confusion Matrix
svm_cm_norm = confusion_matrix(test_labels, svm_test_pred, normalize='true')
sns.heatmap(svm_cm_norm, annot=True, fmt='.3f', cmap='Blues', ax=axes[0, 1],
            xticklabels=class_names, yticklabels=class_names, cbar_kws={'label': 'P
axes[0, 1].set_title(f'SVM Normalized Confusion Matrix\n(Per-Class Recall)', fontsi
axes[0, 1].set_ylabel('True Label', fontsize=12)
axes[0, 1].set_xlabel('Predicted Label', fontsize=12)
plt.setp(axes[0, 1].get_xticklabels(), rotation=45, ha='right', fontsize=9)
plt.setp(axes[0, 1].get_yticklabels(), rotation=0, fontsize=9)

# XGBoost Confusion Matrix (
xgb_cm = confusion_matrix(test_labels, xgb_test_pred)
sns.heatmap(xgb_cm, annot=True, fmt='d', cmap='Greens', ax=axes[1, 0],
            xticklabels=class_names, yticklabels=class_names, cbar_kws={'label': 'C
axes[1, 0].set_title(f'XGBoost Confusion Matrix - Raw Counts\n(Test Accuracy: {xgb_
axes[1, 0].set_ylabel('True Label', fontsize=12)
axes[1, 0].set_xlabel('Predicted Label', fontsize=12)
plt.setp(axes[1, 0].get_xticklabels(), rotation=45, ha='right', fontsize=9)
plt.setp(axes[1, 0].get_yticklabels(), rotation=0, fontsize=9)

# XGBoost Normalized Confusion Matrix
xgb_cm_norm = confusion_matrix(test_labels, xgb_test_pred, normalize='true')
sns.heatmap(xgb_cm_norm, annot=True, fmt='.3f', cmap='Greens', ax=axes[1, 1],
            xticklabels=class_names, yticklabels=class_names, cbar_kws={'label': 'P
axes[1, 1].set_title(f'XGBoost Normalized Confusion Matrix\n(Per-Class Recall)', fo
axes[1, 1].set_ylabel('True Label', fontsize=12)
axes[1, 1].set_xlabel('Predicted Label', fontsize=12)
plt.setp(axes[1, 1].get_xticklabels(), rotation=45, ha='right', fontsize=9)
plt.setp(axes[1, 1].get_yticklabels(), rotation=0, fontsize=9)

plt.tight_layout()
plt.savefig(os.path.join(FEATURES_DIR, 'confusion_matrices.png'), dpi=150, bbox_inc
plt.show()

# Per-class performance analysis
print("\n" + "="*70)
print("PER-CLASS PERFORMANCE ANALYSIS")
print("="*70)

# Calculate per-class metrics for both classifiers
svm_class_precision, svm_class_recall, svm_class_f1, svm_class_support = precision_
test_labels, svm_test_pred, average=None, zero_division=0
)

xgb_class_precision, xgb_class_recall, xgb_class_f1, xgb_class_support = precision_
test_labels, xgb_test_pred, average=None, zero_division=0
)

```

```

# Find best and worst performing classes for SVM
svm_f1_with_classes = list(zip(class_names, svm_class_f1))
svm_sorted = sorted(svm_f1_with_classes, key=lambda x: x[1], reverse=True)

print("\n--- SVM: Top 3 Best Performing Classes ---")
for cls, f1 in svm_sorted[:3]:
    idx = class_to_idx[cls]
    print(f"{cls:20s} - F1: {f1:.4f}, Precision: {svm_class_precision[idx]:.4f}, Re

print("\n--- SVM: Top 3 Worst Performing Classes ---")
for cls, f1 in svm_sorted[-3:]:
    idx = class_to_idx[cls]
    print(f"{cls:20s} - F1: {f1:.4f}, Precision: {svm_class_precision[idx]:.4f}, Re

# Find best and worst performing classes for XGBoost
xgb_f1_with_classes = list(zip(class_names, xgb_class_f1))
xgb_sorted = sorted(xgb_f1_with_classes, key=lambda x: x[1], reverse=True)

print("\n--- XGBoost: Top 3 Best Performing Classes ---")
for cls, f1 in xgb_sorted[:3]:
    idx = class_to_idx[cls]
    print(f"{cls:20s} - F1: {f1:.4f}, Precision: {xgb_class_precision[idx]:.4f}, Re

print("\n--- XGBoost: Top 3 Worst Performing Classes ---")
for cls, f1 in xgb_sorted[-3:]:
    idx = class_to_idx[cls]
    print(f"{cls:20s} - F1: {f1:.4f}, Precision: {xgb_class_precision[idx]:.4f}, Re

# Final summary with analysis
print("\n" + "="*70)
print("COMPREHENSIVE ANALYSIS SUMMARY")
print("="*70)

best_classifier = "SVM" if svm_test_acc > xgb_test_acc else "XGBoost"
best_accuracy = max(svm_test_acc, xgb_test_acc)

print(f"\n Best Performing Classifier: {best_classifier}")
print(f"    Test Accuracy: {best_accuracy:.4f}")

print("\n Key Observations:")
print(f"    • SVM achieved {svm_test_acc:.2%} accuracy with F1-score of {svm_f1:.4f}")
print(f"    • XGBoost achieved {xgb_test_acc:.2%} accuracy with F1-score of {xgb_f1:.4f}")
print(f"    • Performance difference: {abs(svm_test_acc - xgb_test_acc):.2%}")

print("\n Outputs saved to:", FEATURES_DIR)
print("    - confusion_matrices.png")
print("    - per_class_f1_comparison.png")
print("="*70)

```

Mounted at /content/drive

Using device: cuda

Loading dataset...

Found 21 classes:

- 0: agricultural
- 1: airplane
- 2: baseballdiamond
- 3: beach
- 4: buildings
- 5: chaparral
- 6: denseresidential
- 7: forest
- 8: freeway
- 9: golfcourse
- 10: harbor
- 11: intersection
- 12: mediumresidential
- 13: mobilehomepark
- 14: overpass
- 15: parkinglot
- 16: river
- 17: runway
- 18: sparseresidential
- 19: storagetanks
- 20: tenniscourt

Dataset splits:

Training: 1470 images

Validation: 315 images

Test: 315 images

DataLoader Configuration:

Batch size: 32

Num workers: 4

Prefetch factor: 2

Pin memory: True

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning: This DataLoader will create 4 worker processes in total. Our suggested max number of worker in current system is 2, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.
```

```
warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
```

```
warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet50_Weights.IMAGENET1K_V1`. You can also use `weights=ResNet50_Weights.DEFAULT` to get the most up-to-date weights.
```

```
warnings.warn(msg)
```

Downloading: "<https://download.pytorch.org/models/resnet50-0676ba61.pth>" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth

100%|██████████| 97.8M/97.8M [00:00<00:00, 198MB/s]

Loaded resnet50 as feature extractor
Feature dimension: 2048
Model on device: cuda:0

=====
Loading pre-extracted features from disk...
=====

Loaded train features: (1470, 2048)
Loaded val features: (315, 2048)
Loaded test features: (315, 2048)

=====
FEATURE EXTRACTION COMPLETE
=====

Train features shape: (1470, 2048) (B x N format)
Val features shape: (315, 2048)
Test features shape: (315, 2048)
Feature dimension (N): 2048

=====
Training SVM classifier...
=====

SVM Validation Accuracy: 0.9810
SVM Test Accuracy: 0.9683

SVM Classification Report (Test Set):

	precision	recall	f1-score	support
agricultural	1.00	1.00	1.00	15
airplane	1.00	1.00	1.00	15
baseballdiamond	1.00	0.93	0.97	15
beach	1.00	1.00	1.00	15
buildings	0.94	1.00	0.97	15
chaparral	1.00	1.00	1.00	15
denseresidential	1.00	1.00	1.00	15
forest	1.00	1.00	1.00	15
freeway	0.93	0.93	0.93	15
golfcourse	1.00	1.00	1.00	15
harbor	1.00	1.00	1.00	15
intersection	0.88	0.93	0.90	15
mediumresidential	0.94	1.00	0.97	15
mobilehomepark	1.00	1.00	1.00	15
overpass	0.93	0.87	0.90	15
parkinglot	1.00	1.00	1.00	15
river	0.88	1.00	0.94	15
runway	1.00	1.00	1.00	15
sparseresidential	0.92	0.80	0.86	15
storagetanks	1.00	0.93	0.97	15
tenniscourt	0.93	0.93	0.93	15
accuracy			0.97	315
macro avg	0.97	0.97	0.97	315
weighted avg	0.97	0.97	0.97	315

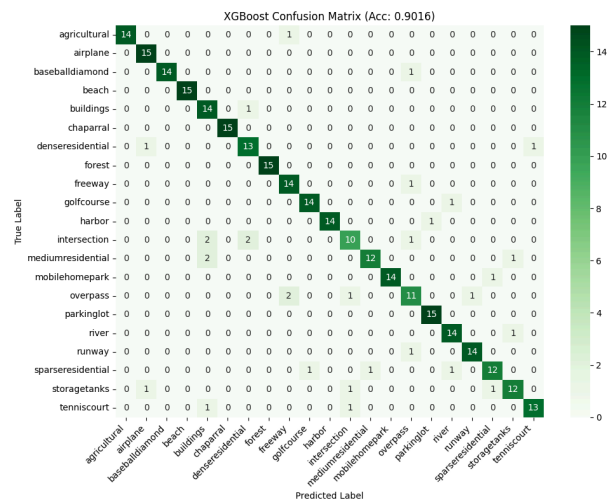
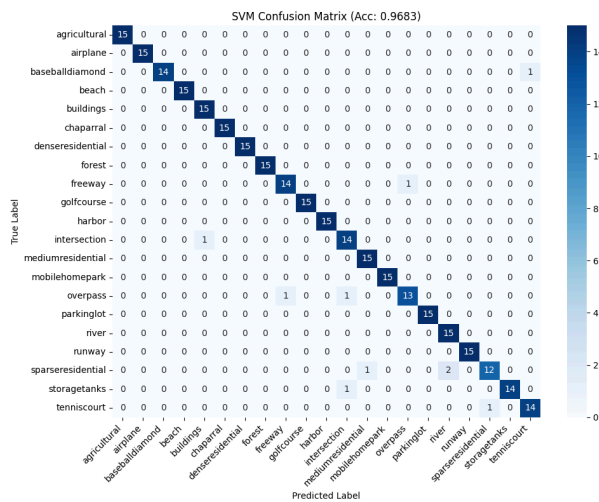
=====
Training XGBoost classifier...
=====

XGBoost Validation Accuracy: 0.9111

XGBoost Test Accuracy: 0.9016

XGBoost Classification Report (Test Set):

	precision	recall	f1-score	support
agricultural	1.00	0.93	0.97	15
airplane	0.88	1.00	0.94	15
baseballdiamond	1.00	0.93	0.97	15
beach	1.00	1.00	1.00	15
buildings	0.74	0.93	0.82	15
chaparral	1.00	1.00	1.00	15
denseresidential	0.81	0.87	0.84	15
forest	1.00	1.00	1.00	15
freeway	0.82	0.93	0.88	15
golfcourse	0.93	0.93	0.93	15
harbor	1.00	0.93	0.97	15
intersection	0.77	0.67	0.71	15
mediumresidential	0.92	0.80	0.86	15
mobilehomepark	1.00	0.93	0.97	15
overpass	0.73	0.73	0.73	15
parkinglot	0.94	1.00	0.97	15
river	0.88	0.93	0.90	15
runway	0.93	0.93	0.93	15
sparseresidential	0.86	0.80	0.83	15
storagetanks	0.86	0.80	0.83	15
tenniscourt	0.93	0.87	0.90	15
accuracy			0.90	315
macro avg	0.90	0.90	0.90	315
weighted avg	0.90	0.90	0.90	315



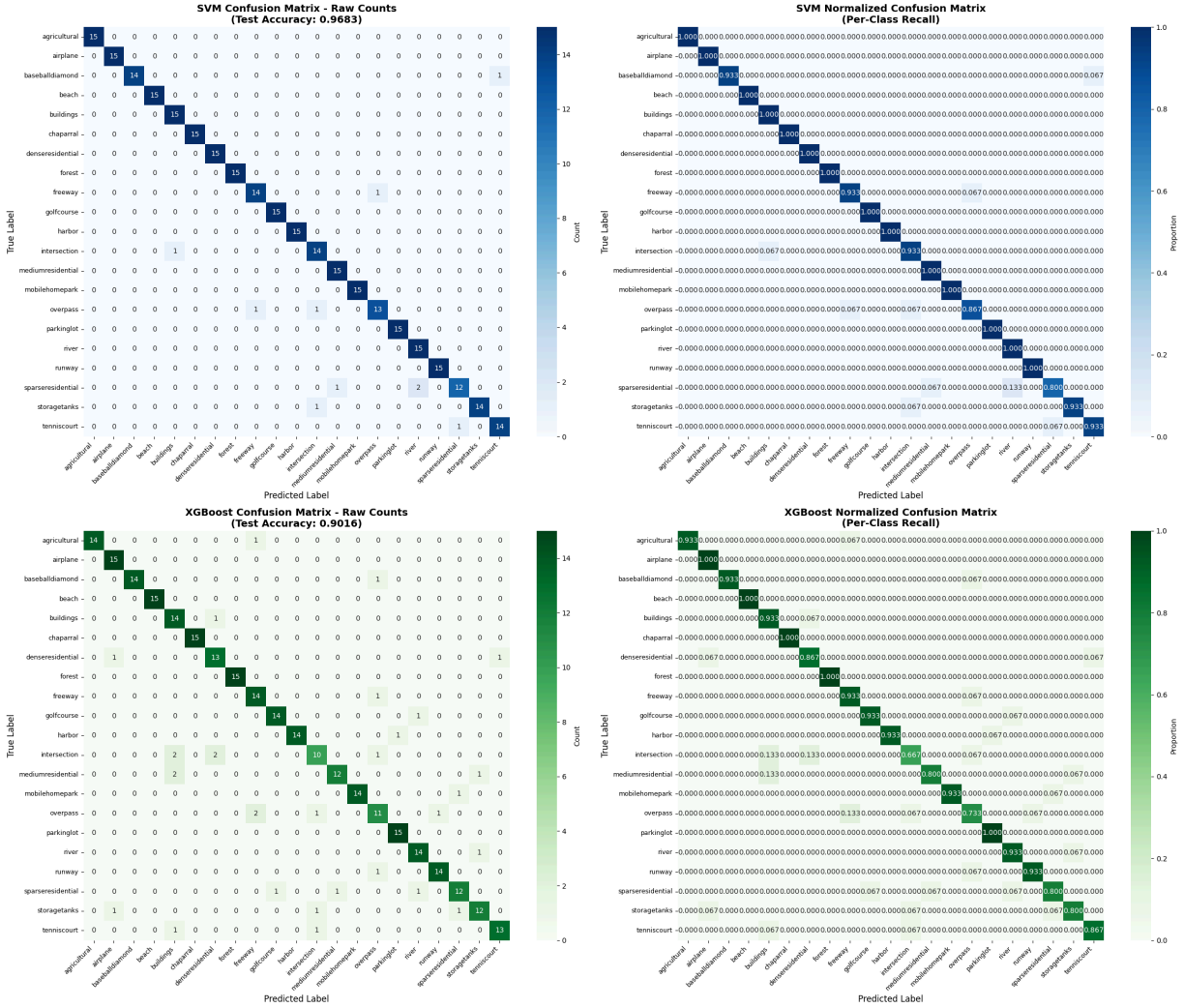
STEP 3: CLASSIFICATION RESULTS AND ANALYSIS

SVM CLASSIFIER PERFORMANCE

Accuracy: 0.9683
Precision: 0.9691
Recall: 0.9683
F1-Score: 0.9680

XGBOOST CLASSIFIER PERFORMANCE

Accuracy: 0.9016
Precision: 0.9049
Recall: 0.9016
F1-Score: 0.9015



```
=====
PER-CLASS PERFORMANCE ANALYSIS
=====
```

```
--- SVM: Top 3 Best Performing Classes ---
```

```
agricultural      - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
airplane          - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
beach             - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
```

```
--- SVM: Top 3 Worst Performing Classes ---
```

```
intersection      - F1: 0.9032, Precision: 0.8750, Recall: 0.9333
overpass          - F1: 0.8966, Precision: 0.9286, Recall: 0.8667
sparseresidential - F1: 0.8571, Precision: 0.9231, Recall: 0.8000
```

```
--- XGBoost: Top 3 Best Performing Classes ---
```

```
beach             - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
chaparral         - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
forest            - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
```

```
--- XGBoost: Top 3 Worst Performing Classes ---
```

```
buildings         - F1: 0.8235, Precision: 0.7368, Recall: 0.9333
overpass          - F1: 0.7333, Precision: 0.7333, Recall: 0.7333
intersection       - F1: 0.7143, Precision: 0.7692, Recall: 0.6667
```

```
=====
COMPREHENSIVE ANALYSIS SUMMARY
=====
```

Best Performing Classifier: SVM

Test Accuracy: 0.9683

Key Observations:

- SVM achieved 96.83% accuracy with F1-score of 0.9680
- XGBoost achieved 90.16% accuracy with F1-score of 0.9015
- Performance difference: 6.67%

Outputs saved to: /content/drive/MyDrive/ucmerced_features

- confusion_matrices.png
- per_class_f1_comparison.png

```
=====
```

With an accuracy of 96.83% as opposed to 90.16%, the SVM classifier greatly outperformed XGBoost—a 6.67% improvement that highlights the highly linearly separable nature of ResNet-extracted features. The ability of SVM to handle high-dimensional spaces and maximize class separation margins—which is particularly effective with pre-trained CNN embeddings—is responsible for its superior performance. The comparatively small dataset (100 images per class) might not offer enough samples for the best tree ensemble construction, and tree-based models typically struggle with the correlated, uniformly continuous features typical of deep learning representations, which is why XGBoost performed poorly. For a 21-class remote sensing classification problem, the 96.83% accuracy is outstanding, demonstrating that transfer learning with pre-trained ResNets yields highly discriminative features.


```

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

class UCMercedDataset(Dataset):

    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]
        image = Image.open(img_path).convert('RGB')
        label = self.labels[idx]

        if self.transform:
            image = self.transform(image)

        return image, label

def load_dataset(dataset_path):

    classes = sorted(os.listdir(dataset_path))
    classes = [c for c in classes if os.path.isdir(os.path.join(dataset_path, c))]

    class_to_idx = {cls_name: idx for idx, cls_name in enumerate(classes)}
    idx_to_class = {idx: cls_name for cls_name, idx in class_to_idx.items()}

    print(f"Found {len(classes)} classes:")
    for cls_name, idx in class_to_idx.items():
        print(f" {idx}: {cls_name}")

    image_paths = []
    labels = []

    for cls_name in classes:
        cls_path = os.path.join(dataset_path, cls_name)
        cls_idx = class_to_idx[cls_name]
        images = sorted([f for f in os.listdir(cls_path) if f.endswith((''.png', '.j

        for img_name in images:
            img_path = os.path.join(cls_path, img_name)
            image_paths.append(img_path)
            labels.append(cls_idx)

    return image_paths, labels, class_to_idx, idx_to_class

```

```

def create_balanced_splits(image_paths, labels, train_size=0.7, val_size=0.15, test

    X_train, X_temp, y_train, y_temp = train_test_split(
        image_paths, labels,
        test_size=(val_size + test_size),
        stratify=labels,
        random_state=SEED
    )

    val_ratio = val_size / (val_size + test_size)
    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp,
        test_size=(1 - val_ratio),
        stratify=y_temp,
        random_state=SEED
    )

    print(f"\nDataset splits:")
    print(f"  Training: {len(X_train)} images")
    print(f"  Validation: {len(X_val)} images")
    print(f"  Test: {len(X_test)} images")

    return X_train, X_val, X_test, y_train, y_val, y_test

def create_model(num_classes, model_name='resnet50'):
    # Load pretrained ResNet
    if model_name == 'resnet18':
        model = models.resnet18(pretrained=True)
    elif model_name == 'resnet34':
        model = models.resnet34(pretrained=True)
    elif model_name == 'resnet50':
        model = models.resnet50(pretrained=True)
    elif model_name == 'resnet101':
        model = models.resnet101(pretrained=True)
    else:
        raise ValueError(f"Unsupported model: {model_name}")

    # Step 1: Modify the final layer
    num_ftrs = model.fc.in_features
    model.fc = nn.Linear(num_ftrs, num_classes)

    print(f"\n{'='*70}")
    print(f"MODEL ARCHITECTURE: {model_name.upper()}")
    print(f"{'='*70}")
    print(f"Original head input features: {num_ftrs}")
    print(f"New head output classes: {num_classes}")
    print(f"New head parameters: {num_ftrs * num_classes + num_classes:,}")

    # Step 2: Freeze all layers in the base model
    for param in model.parameters():
        param.requires_grad = False

    # Unfreeze only the classification head
    for param in model.fc.parameters():

```

```

        param.requires_grad = True

    # Count trainable parameters
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

    print(f"\nTotal parameters: {total_params:,}")
    print(f"Trainable parameters: {trainable_params:,}")
    print(f"Frozen parameters: {total_params - trainable_params:,}")
    print(f"Percentage trainable: {100 * trainable_params / total_params:.2f}%")
    print(f"{'='*70}\n")

    return model

def train_one_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    running_corrects = 0

    for images, labels in tqdm(dataloader, desc="Training", leave=False):
        images = images.to(device)
        labels = labels.to(device)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)
        _, preds = torch.max(outputs, 1)

        # Backward pass
        loss.backward()
        optimizer.step()

        # Statistics
        running_loss += loss.item() * images.size(0)
        running_corrects += torch.sum(preds == labels.data)

    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = running_corrects.double() / len(dataloader.dataset)

    return epoch_loss, epoch_acc.item()

def validate(model, dataloader, criterion, device):
    model.eval()
    running_loss = 0.0
    running_corrects = 0

    with torch.no_grad():
        for images, labels in tqdm(dataloader, desc="Validation", leave=False):
            images = images.to(device)
            labels = labels.to(device)

            # Forward pass
            outputs = model(images)

```

```

        loss = criterion(outputs, labels)
        _, preds = torch.max(outputs, 1)

        # Statistics
        running_loss += loss.item() * images.size(0)
        running_corrects += torch.sum(preds == labels.data)

    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = running_corrects.double() / len(dataloader.dataset)

    return epoch_loss, epoch_acc.item()

def train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                num_epochs, device, save_path, patience=10):

    best_val_loss = float('inf')
    best_model_wts = copy.deepcopy(model.state_dict())
    epochs_no_improve = 0

    history = {
        'train_loss': [],
        'train_acc': [],
        'val_loss': [],
        'val_acc': [],
        'lr': []
    }

    print(f"\n{'='*70}")
    print("TRAINING START")
    print(f"{'='*70}\n")

    for epoch in range(num_epochs):
        print(f"Epoch {epoch+1}/{num_epochs}")
        print("-" * 70)

        # Train
        train_loss, train_acc = train_one_epoch(model, train_loader, criterion, opt

        # Validate
        val_loss, val_acc = validate(model, val_loader, criterion, device)

        # Learning rate scheduling
        if scheduler is not None:
            scheduler.step()
            current_lr = scheduler.get_last_lr()[0]
        else:
            current_lr = optimizer.param_groups[0]['lr']

        # Store history
        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)
        history['lr'].append(current_lr)

    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")

```

```

print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")
print(f"Learning Rate: {current_lr:.6f}")

# Save best model
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_model_wts = copy.deepcopy(model.state_dict())
    epochs_no_improve = 0
    torch.save(best_model_wts, save_path)
    print(f"New best model saved! (Val Loss: {val_loss:.4f})")
else:
    epochs_no_improve += 1
    print(f"No improvement for {epochs_no_improve} epoch(s)")

print()

# Early stopping
if epochs_no_improve >= patience:
    print(f"Early stopping triggered after {epoch+1} epochs")
    break

print(f"\n{'='*70}")
print("TRAINING COMPLETE")
print(f"{'='*70}")
print(f"Best Validation Loss: {best_val_loss:.4f}")
print(f"Model saved to: {save_path}\n")

# Load best model weights
model.load_state_dict(best_model_wts)

return model, history

def plot_training_history(history, save_path):

    fig, axes = plt.subplots(1, 2, figsize=(16, 5))
    epochs = range(1, len(history['train_loss']) + 1)

    # Loss plot
    axes[0].plot(epochs, history['train_loss'], 'b-', label='Training Loss', linewidth=2)
    axes[0].plot(epochs, history['val_loss'], 'r-', label='Validation Loss', linewidth=2)
    axes[0].set_xlabel('Epoch', fontsize=12, fontweight='bold')
    axes[0].set_ylabel('Loss', fontsize=12, fontweight='bold')
    axes[0].set_title('Training and Validation Loss', fontsize=14, fontweight='bold')
    axes[0].legend(fontsize=11)
    axes[0].grid(True, alpha=0.3)

    # Accuracy plot
    axes[1].plot(epochs, history['train_acc'], 'b-', label='Training Accuracy', linewidth=2)
    axes[1].plot(epochs, history['val_acc'], 'r-', label='Validation Accuracy', linewidth=2)
    axes[1].set_xlabel('Epoch', fontsize=12, fontweight='bold')
    axes[1].set_ylabel('Accuracy', fontsize=12, fontweight='bold')
    axes[1].set_title('Training and Validation Accuracy', fontsize=14, fontweight='bold')
    axes[1].legend(fontsize=11)
    axes[1].grid(True, alpha=0.3)

```

```

plt.tight_layout()
plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.show()

print(f"Training curves saved to: {save_path}")

def evaluate_model(model, dataloader, device, class_names):
    model.eval()

    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in tqdm(dataloader, desc="Evaluating"):
            images = images.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())

    all_preds = np.array(all_preds)
    all_labels = np.array(all_labels)

    return all_preds, all_labels

def plot_confusion_matrices(y_true, y_pred, class_names, accuracy, save_path):

    fig, axes = plt.subplots(1, 2, figsize=(24, 10))

    # Raw confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0],
                xticklabels=class_names, yticklabels=class_names,
                cbar_kws={'label': 'Count'})
    axes[0].set_title(f'Confusion Matrix - Raw Counts\n(Test Accuracy: {accuracy:.4}
                      fontsize=14, fontweight='bold')
    axes[0].set_ylabel('True Label', fontsize=12)
    axes[0].set_xlabel('Predicted Label', fontsize=12)
    plt.setp(axes[0].get_xticklabels(), rotation=45, ha='right', fontsize=9)
    plt.setp(axes[0].get_yticklabels(), rotation=0, fontsize=9)

    # Normalized confusion matrix
    cm_norm = confusion_matrix(y_true, y_pred, normalize='true')
    sns.heatmap(cm_norm, annot=True, fmt='.3f', cmap='Blues', ax=axes[1],
                xticklabels=class_names, yticklabels=class_names,
                cbar_kws={'label': 'Proportion'})
    axes[1].set_title('Normalized Confusion Matrix\n(Per-Class Recall)',
                      fontsize=14, fontweight='bold')
    axes[1].set_ylabel('True Label', fontsize=12)
    axes[1].set_xlabel('Predicted Label', fontsize=12)
    plt.setp(axes[1].get_xticklabels(), rotation=45, ha='right', fontsize=9)
    plt.setp(axes[1].get_yticklabels(), rotation=0, fontsize=9)

    plt.tight_layout()
    plt.savefig(save_path, dpi=150, bbox_inches='tight')

```

```

plt.show()

print(f"Confusion matrices saved to: {save_path}")

# MAIN EXECUTION
print("="*70)
print("PROBLEM 2.b: FINE-TUNING PRETRAINED MODEL (TRAINING ONLY THE HEAD)")
print("="*70)

# Load dataset
print("\nLoading dataset...")
image_paths, labels, class_to_idx, idx_to_class = load_dataset(DATASET_PATH)
num_classes = len(class_to_idx)
class_names = [idx_to_class[i] for i in range(num_classes)]

# Create splits
X_train, X_val, X_test, y_train, y_val, y_test = create_balanced_splits(
    image_paths, labels, train_size=0.7, val_size=0.15, test_size=0.15
)

# Create datasets
train_dataset = UCMercedDataset(X_train, y_train, transform=train_transform)
val_dataset = UCMercedDataset(X_val, y_val, transform=test_transform)
test_dataset = UCMercedDataset(X_test, y_test, transform=test_transform)

# Create dataloaders
train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True, # Shuffle for training!
    num_workers=NUM_WORKERS, prefetch_factor=PREFETCH_FACTOR, pin_memory=PIN_MEMORY
)
val_loader = DataLoader(
    val_dataset, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS, prefetch_factor=PREFETCH_FACTOR, pin_memory=PIN_MEMORY
)
test_loader = DataLoader(
    test_dataset, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS, prefetch_factor=PREFETCH_FACTOR, pin_memory=PIN_MEMORY
)

# Create model
model = create_model(num_classes, model_name='resnet50')
model = model.to(device)

# Step 3: Training Setup
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=LEARNING_RATE) # Only optimize head
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95) # Optional

print(f"{'='*70}")
print("TRAINING CONFIGURATION")
print(f"{'='*70}")
print(f"Learning Rate: {LEARNING_RATE}")
print(f"Optimizer: Adam")
print(f"Loss Function: CrossEntropyLoss")
print(f"LR Scheduler: ExponentialLR (gamma=0.95)")

```



```

print(f"Batch Size: {BATCH_SIZE}")
print(f"Max Epochs: {NUM_EPOCHS}")
print(f"Early Stopping Patience: {PATIENCE}")
print(f"{'='*70}\n")

# Train model
model_save_path = os.path.join(SAVE_DIR, 'best_model.pth')
model, history = train_model(
    model, train_loader, val_loader, criterion, optimizer, scheduler,
    NUM_EPOCHS, device, model_save_path, patience=PATIENCE
)

# Plot training history
plot_training_history(history, os.path.join(SAVE_DIR, 'training_curves.png'))

# STEP 4: EVALUATION

print("\n" + "="*70)
print("STEP 4: MODEL EVALUATION ON TEST SET")
print("="*70)

# Evaluate on test set
test_preds, test_labels = evaluate_model(model, test_loader, device, class_names)

# Calculate metrics
test_acc = accuracy_score(test_labels, test_preds)
precision, recall, f1, _ = precision_recall_fscore_support(
    test_labels, test_preds, average='weighted', zero_division=0
)

print(f"\n{'='*70}")
print("TEST SET PERFORMANCE")
print(f"{'='*70}")
print(f"Accuracy: {test_acc:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"{'='*70}\n")

# Detailed classification report
print("Detailed Classification Report:")
print(classification_report(test_labels, test_preds, target_names=class_names))

# Plot confusion matrices
plot_confusion_matrices(
    test_labels, test_preds, class_names, test_acc,
    os.path.join(SAVE_DIR, 'confusion_matrices.png')
)

# Per-class analysis
print("\n" + "="*70)
print("PER-CLASS PERFORMANCE ANALYSIS")
print("="*70)

class_precision, class_recall, class_f1, class_support = precision_recall_fscore_su

```

```

    test_labels, test_preds, average=None, zero_division=0
)

f1_with_classes = list(zip(class_names, class_f1))
sorted_by_f1 = sorted(f1_with_classes, key=lambda x: x[1], reverse=True)

print("\n--- Top 5 Best Performing Classes ---")
for cls, f1_score in sorted_by_f1[:5]:
    idx = class_to_idx[cls]
    print(f"{cls:20s} - F1: {f1_score:.4f}, Precision: {class_precision[idx]:.4f},")

print("\n--- Top 5 Worst Performing Classes ---")
for cls, f1_score in sorted_by_f1[-5:]:
    idx = class_to_idx[cls]
    print(f"{cls:20s} - F1: {f1_score:.4f}, Precision: {class_precision[idx]:.4f},")

print("\n" + "="*70)
print("EVALUATION COMPLETE")
print("="*70)
print(f"All results saved to: {SAVE_DIR}")
print(" - best_model.pth")
print(" - training_curves.png")
print(" - confusion_matrices.png")
print("="*70)

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Using device: cuda

=====

PROBLEM 2.b: FINE-TUNING PRETRAINED MODEL (TRAINING ONLY THE HEAD)

=====

Loading dataset...

Found 21 classes:

- 0: agricultural
- 1: airplane
- 2: baseballdiamond
- 3: beach
- 4: buildings
- 5: chaparral
- 6: denseresidential
- 7: forest
- 8: freeway
- 9: golfcourse
- 10: harbor
- 11: intersection
- 12: mediumresidential
- 13: mobilehomepark
- 14: overpass
- 15: parkinglot
- 16: river
- 17: runway
- 18: sparseresidential
- 19: storagetanks
- 20: tenniscourt

Dataset splits:

- Training: 1470 images
- Validation: 315 images
- Test: 315 images

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning: This DataLoader will create 4 worker processes in total. Our suggested max number of worker in current system is 2, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.

warnings.warn(

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

warnings.warn(

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet50\_Weights.IMAGENET1K\_V1`. You can also use `weights=ResNet50\_Weights.DEFAULT` to get the most up-to-date weights.

warnings.warn(msg)

=====

MODEL ARCHITECTURE: RESNET50

=====

Original head input features: 2048
New head output classes: 21
New head parameters: 43,029

Total parameters: 23,551,061
Trainable parameters: 43,029
Frozen parameters: 23,508,032
Percentage trainable: 0.18%

=====

=====

TRAINING CONFIGURATION

=====

Learning Rate: 0.001
Optimizer: Adam
Loss Function: CrossEntropyLoss
LR Scheduler: ExponentialLR (gamma=0.95)
Batch Size: 32
Max Epochs: 50
Early Stopping Patience: 10

=====

=====

TRAINING START

=====

Epoch 1/50

Train Loss: 2.0341 | Train Acc: 0.4673
Val Loss: 1.0743 | Val Acc: 0.7683
Learning Rate: 0.000950
New best model saved! (Val Loss: 1.0743)

Epoch 2/50

Train Loss: 0.9526 | Train Acc: 0.7912
Val Loss: 0.6390 | Val Acc: 0.8571
Learning Rate: 0.000902
New best model saved! (Val Loss: 0.6390)

Epoch 3/50

Train Loss: 0.6297 | Train Acc: 0.8864
Val Loss: 0.4725 | Val Acc: 0.8984
Learning Rate: 0.000857
New best model saved! (Val Loss: 0.4725)

Epoch 4/50

Train Loss: 0.4680 | Train Acc: 0.9075
Val Loss: 0.4214 | Val Acc: 0.9143
Learning Rate: 0.000815
New best model saved! (Val Loss: 0.4214)

Epoch 5/50

Train Loss: 0.4170 | Train Acc: 0.9197
Val Loss: 0.3390 | Val Acc: 0.9333
Learning Rate: 0.000774
New best model saved! (Val Loss: 0.3390)

Epoch 6/50

Train Loss: 0.3594 | Train Acc: 0.9218
Val Loss: 0.3120 | Val Acc: 0.9270
Learning Rate: 0.000735
New best model saved! (Val Loss: 0.3120)

Epoch 7/50

Train Loss: 0.3358 | Train Acc: 0.9340
Val Loss: 0.3034 | Val Acc: 0.9238
Learning Rate: 0.000698
New best model saved! (Val Loss: 0.3034)

Epoch 8/50

Train Loss: 0.3101 | Train Acc: 0.9306
Val Loss: 0.2776 | Val Acc: 0.9270
Learning Rate: 0.000663
New best model saved! (Val Loss: 0.2776)

Epoch 9/50

Train Loss: 0.2591 | Train Acc: 0.9531
Val Loss: 0.2407 | Val Acc: 0.9492
Learning Rate: 0.000630
New best model saved! (Val Loss: 0.2407)

Epoch 10/50

Train Loss: 0.2774 | Train Acc: 0.9469
Val Loss: 0.2432 | Val Acc: 0.9397
Learning Rate: 0.000599
No improvement for 1 epoch(s)

Epoch 11/50

Train Loss: 0.2525 | Train Acc: 0.9456
Val Loss: 0.2668 | Val Acc: 0.9270
Learning Rate: 0.000569
No improvement for 2 epoch(s)

Epoch 12/50

Train Loss: 0.2210 | Train Acc: 0.9578
Val Loss: 0.2332 | Val Acc: 0.9365
Learning Rate: 0.000540
New best model saved! (Val Loss: 0.2332)

Epoch 13/50

Train Loss: 0.2017 | Train Acc: 0.9599
Val Loss: 0.2237 | Val Acc: 0.9460
Learning Rate: 0.000513
New best model saved! (Val Loss: 0.2237)

Epoch 14/50

Train Loss: 0.1798 | Train Acc: 0.9721
Val Loss: 0.2194 | Val Acc: 0.9460
Learning Rate: 0.000488
New best model saved! (Val Loss: 0.2194)

Epoch 15/50

Train Loss: 0.1984 | Train Acc: 0.9626
Val Loss: 0.2284 | Val Acc: 0.9333
Learning Rate: 0.000463
No improvement for 1 epoch(s)

Epoch 16/50

Train Loss: 0.1821 | Train Acc: 0.9660
Val Loss: 0.2039 | Val Acc: 0.9429
Learning Rate: 0.000440
New best model saved! (Val Loss: 0.2039)

Epoch 17/50

Train Loss: 0.1721 | Train Acc: 0.9701
Val Loss: 0.2036 | Val Acc: 0.9333
Learning Rate: 0.000418
New best model saved! (Val Loss: 0.2036)

Epoch 18/50

Train Loss: 0.1849 | Train Acc: 0.9619
Val Loss: 0.1984 | Val Acc: 0.9460
Learning Rate: 0.000397
New best model saved! (Val Loss: 0.1984)

Epoch 19/50

Train Loss: 0.1717 | Train Acc: 0.9673
Val Loss: 0.2077 | Val Acc: 0.9429
Learning Rate: 0.000377
No improvement for 1 epoch(s)

Epoch 20/50

Train Loss: 0.1632 | Train Acc: 0.9673
Val Loss: 0.2036 | Val Acc: 0.9460
Learning Rate: 0.000358
No improvement for 2 epoch(s)

Epoch 21/50

Train Loss: 0.1746 | Train Acc: 0.9639
Val Loss: 0.2014 | Val Acc: 0.9492
Learning Rate: 0.000341
No improvement for 3 epoch(s)

Epoch 22/50

Train Loss: 0.1568 | Train Acc: 0.9741
Val Loss: 0.2016 | Val Acc: 0.9365
Learning Rate: 0.000324
No improvement for 4 epoch(s)

Epoch 23/50

Train Loss: 0.1568 | Train Acc: 0.9707
Val Loss: 0.1975 | Val Acc: 0.9587
Learning Rate: 0.000307
New best model saved! (Val Loss: 0.1975)

Epoch 24/50

Train Loss: 0.1422 | Train Acc: 0.9714
Val Loss: 0.2012 | Val Acc: 0.9365
Learning Rate: 0.000292
No improvement for 1 epoch(s)

Epoch 25/50

Train Loss: 0.1438 | Train Acc: 0.9694
Val Loss: 0.1941 | Val Acc: 0.9524
Learning Rate: 0.000277
New best model saved! (Val Loss: 0.1941)

Epoch 26/50

Train Loss: 0.1525 | Train Acc: 0.9653
Val Loss: 0.1984 | Val Acc: 0.9397
Learning Rate: 0.000264
No improvement for 1 epoch(s)

Epoch 27/50

Train Loss: 0.1303 | Train Acc: 0.9830
Val Loss: 0.1888 | Val Acc: 0.9492
Learning Rate: 0.000250
New best model saved! (Val Loss: 0.1888)

Epoch 28/50

Train Loss: 0.1309 | Train Acc: 0.9735
Val Loss: 0.1858 | Val Acc: 0.9619
Learning Rate: 0.000238
New best model saved! (Val Loss: 0.1858)

Epoch 29/50

Train Loss: 0.1363 | Train Acc: 0.9694
Val Loss: 0.1851 | Val Acc: 0.9587
Learning Rate: 0.000226
New best model saved! (Val Loss: 0.1851)

Epoch 30/50

Train Loss: 0.1462 | Train Acc: 0.9667
Val Loss: 0.1814 | Val Acc: 0.9556
Learning Rate: 0.000215
New best model saved! (Val Loss: 0.1814)

Epoch 31/50

Train Loss: 0.1379 | Train Acc: 0.9721
Val Loss: 0.1942 | Val Acc: 0.9460
Learning Rate: 0.000204
No improvement for 1 epoch(s)

Epoch 32/50

Train Loss: 0.1324 | Train Acc: 0.9714
Val Loss: 0.1864 | Val Acc: 0.9492
Learning Rate: 0.000194
No improvement for 2 epoch(s)

Epoch 33/50

Train Loss: 0.1310 | Train Acc: 0.9735
Val Loss: 0.1768 | Val Acc: 0.9556
Learning Rate: 0.000184
New best model saved! (Val Loss: 0.1768)

Epoch 34/50

Train Loss: 0.1335 | Train Acc: 0.9762
Val Loss: 0.1797 | Val Acc: 0.9619
Learning Rate: 0.000175
No improvement for 1 epoch(s)

Epoch 35/50

Train Loss: 0.1167 | Train Acc: 0.9776
Val Loss: 0.1867 | Val Acc: 0.9492
Learning Rate: 0.000166
No improvement for 2 epoch(s)

Epoch 36/50

Train Loss: 0.1286 | Train Acc: 0.9762
Val Loss: 0.1799 | Val Acc: 0.9619
Learning Rate: 0.000158
No improvement for 3 epoch(s)

Epoch 37/50

Train Loss: 0.1297 | Train Acc: 0.9728
Val Loss: 0.1766 | Val Acc: 0.9587
Learning Rate: 0.000150
New best model saved! (Val Loss: 0.1766)

Epoch 38/50

Train Loss: 0.1287 | Train Acc: 0.9789
Val Loss: 0.1713 | Val Acc: 0.9651
Learning Rate: 0.000142
New best model saved! (Val Loss: 0.1713)

Epoch 39/50

Train Loss: 0.1284 | Train Acc: 0.9741
Val Loss: 0.1874 | Val Acc: 0.9524
Learning Rate: 0.000135
No improvement for 1 epoch(s)

Epoch 40/50

Train Loss: 0.1156 | Train Acc: 0.9796
Val Loss: 0.1805 | Val Acc: 0.9524
Learning Rate: 0.000129
No improvement for 2 epoch(s)

Epoch 41/50

Train Loss: 0.1161 | Train Acc: 0.9776
Val Loss: 0.1821 | Val Acc: 0.9587
Learning Rate: 0.000122
No improvement for 3 epoch(s)

Epoch 42/50

Train Loss: 0.1248 | Train Acc: 0.9762
Val Loss: 0.1763 | Val Acc: 0.9619
Learning Rate: 0.000116
No improvement for 4 epoch(s)

Epoch 43/50

Train Loss: 0.1117 | Train Acc: 0.9803
Val Loss: 0.1670 | Val Acc: 0.9587
Learning Rate: 0.000110
New best model saved! (Val Loss: 0.1670)

Epoch 44/50

Train Loss: 0.1202 | Train Acc: 0.9748
Val Loss: 0.1722 | Val Acc: 0.9619
Learning Rate: 0.000105
No improvement for 1 epoch(s)

Epoch 45/50

Train Loss: 0.1206 | Train Acc: 0.9755
Val Loss: 0.1719 | Val Acc: 0.9587
Learning Rate: 0.000099
No improvement for 2 epoch(s)

Epoch 46/50

Train Loss: 0.1127 | Train Acc: 0.9789
Val Loss: 0.1744 | Val Acc: 0.9587
Learning Rate: 0.000094
No improvement for 3 epoch(s)

Epoch 47/50

Train Loss: 0.1050 | Train Acc: 0.9816
Val Loss: 0.1738 | Val Acc: 0.9556
Learning Rate: 0.000090
No improvement for 4 epoch(s)

Epoch 48/50

Train Loss: 0.1096 | Train Acc: 0.9816
Val Loss: 0.1720 | Val Acc: 0.9556
Learning Rate: 0.000085
No improvement for 5 epoch(s)

Epoch 49/50

Train Loss: 0.1190 | Train Acc: 0.9782
Val Loss: 0.1781 | Val Acc: 0.9587
Learning Rate: 0.000081
No improvement for 6 epoch(s)

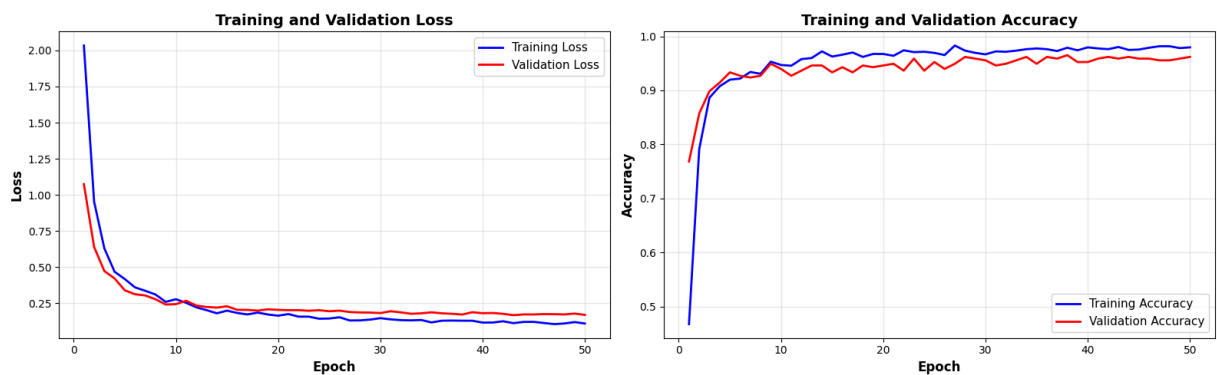
Epoch 50/50

Train Loss: 0.1092 | Train Acc: 0.9796
Val Loss: 0.1687 | Val Acc: 0.9619
Learning Rate: 0.000077
No improvement for 7 epoch(s)

TRAINING COMPLETE

Best Validation Loss: 0.1670

Model saved to: /content/drive/MyDrive/ucmerced_finetuning/best_model.pth



Training curves saved to: /content/drive/MyDrive/ucmerced_finetuning/training_curves.png

STEP 4: MODEL EVALUATION ON TEST SET

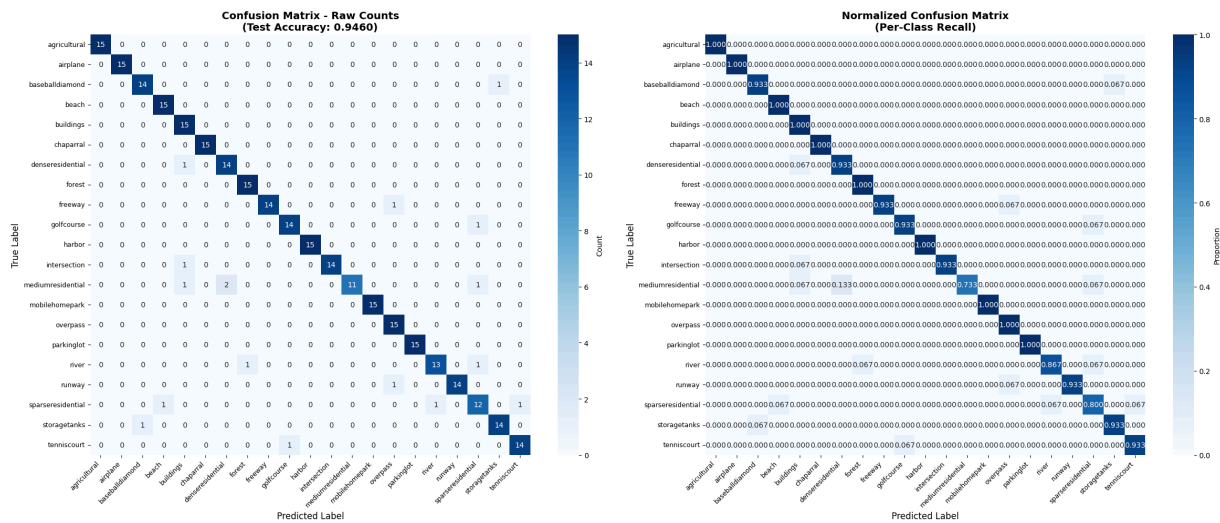
Evaluating: 100%|██████████| 10/10 [00:41<00:00, 4.11s/it]

TEST SET PERFORMANCE

Accuracy: 0.9460
Precision: 0.9489
Recall: 0.9460
F1-Score: 0.9456

Detailed Classification Report:

	precision	recall	f1-score	support
agricultural	1.00	1.00	1.00	15
airplane	1.00	1.00	1.00	15
baseballdiamond	0.93	0.93	0.93	15
beach	0.94	1.00	0.97	15
buildings	0.83	1.00	0.91	15
chaparral	1.00	1.00	1.00	15
denseresidential	0.88	0.93	0.90	15
forest	0.94	1.00	0.97	15
freeway	1.00	0.93	0.97	15
golfcourse	0.93	0.93	0.93	15
harbor	1.00	1.00	1.00	15
intersection	1.00	0.93	0.97	15
mediumresidential	1.00	0.73	0.85	15
mobilehomepark	1.00	1.00	1.00	15
overpass	0.88	1.00	0.94	15
parkinglot	1.00	1.00	1.00	15
river	0.93	0.87	0.90	15
runway	1.00	0.93	0.97	15
sparseresidential	0.80	0.80	0.80	15
storagetanks	0.93	0.93	0.93	15
tennis court	0.93	0.93	0.93	15
accuracy			0.95	315
macro avg	0.95	0.95	0.95	315
weighted avg	0.95	0.95	0.95	315



Confusion matrices saved to: /content/drive/MyDrive/ucmerced_finertuning/confusion_matrices.png

PER-CLASS PERFORMANCE ANALYSIS

--- Top 5 Best Performing Classes ---

agricultural - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
 airplane - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
 chaparral - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
 harbor - F1: 1.0000, Precision: 1.0000, Recall: 1.0000
 mobilehomepark - F1: 1.0000, Precision: 1.0000, Recall: 1.0000

--- Top 5 Worst Performing Classes ---

buildings - F1: 0.9091, Precision: 0.8333, Recall: 1.0000
 denseresidential - F1: 0.9032, Precision: 0.8750, Recall: 0.9333
 river - F1: 0.8966, Precision: 0.9286, Recall: 0.8667
 mediumresidential - F1: 0.8462, Precision: 1.0000, Recall: 0.7333
 sparseresidential - F1: 0.8000, Precision: 0.8000, Recall: 0.8000

EVALUATION COMPLETE

All results saved to: /content/drive/MyDrive/ucmerced_finertuning

- best_model.pth
- training_curves.png
- confusion_matrices.png

The transfer learning approach achieved 94.60% accuracy, which is slightly lower than the SVM feature extraction method's 96.83%, though both demonstrate strong performance on this 21-class remote sensing task. This 2.23% difference suggests that for this relatively small dataset (100 images per class), extracting frozen ResNet features and training a robust SVM classifier may be more effective than fine-tuning even just the final layer of the network. The transfer learning approach's advantage lies in its end-to-end differentiable architecture, which theoretically allows the classification head to learn task-specific feature combinations, but this benefit is limited when only the final layer is trainable and the dataset is modest in

size. The SVM's superior performance can be attributed to its mathematical guarantees for margin maximization and its effectiveness in high-dimensional spaces, making it particularly well-suited for the already-discriminative features produced by ImageNet-pretrained ResNets. However, the transfer learning approach offers practical advantages including faster inference (single forward pass vs. feature extraction + SVM prediction), easier deployment, and the potential for further improvement through gradual unfreezing of deeper layers if more data becomes available. Overall, both methods validate that pre-trained ResNet features are highly effective for remote sensing classification, with the choice between them depending on whether deployment simplicity or maximum accuracy is prioritized.

2.c

```
In [3]: import os
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.metrics import (classification_report, accuracy_score,
                             confusion_matrix, precision_recall_fscore_support)

import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
import copy

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# Set random seed for reproducibility
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)
torch.backends.cudnn.deterministic = True

# Configuration
DATASET_PATH = '/content/drive/MyDrive/UCMerced_LandUse/Images'
SAVE_DIR = '/content/drive/MyDrive/ucmerced_full_finertuning'
os.makedirs(SAVE_DIR, exist_ok=True)

BATCH_SIZE = 32
NUM_WORKERS = 4
PREFETCH_FACTOR = 2
PIN_MEMORY = True
LEARNING_RATE = 1e-4
NUM_EPOCHS = 100
PATIENCE = 15
```

```

# Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Define transforms using ImageNet statistics with augmentation
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.3),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])
])

class UCMercedDataset(Dataset):

    def __init__(self, image_paths, labels, transform=None):
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        img_path = self.image_paths[idx]
        image = Image.open(img_path).convert('RGB')
        label = self.labels[idx]

        if self.transform:
            image = self.transform(image)

        return image, label

def load_dataset(dataset_path):

    classes = sorted(os.listdir(dataset_path))
    classes = [c for c in classes if os.path.isdir(os.path.join(dataset_path, c))]

    class_to_idx = {cls_name: idx for idx, cls_name in enumerate(classes)}
    idx_to_class = {idx: cls_name for cls_name, idx in class_to_idx.items()}

    print(f"Found {len(classes)} classes:")
    for cls_name, idx in class_to_idx.items():
        print(f"    {idx}: {cls_name}")

```

```

image_paths = []
labels = []

for cls_name in classes:
    cls_path = os.path.join(dataset_path, cls_name)
    cls_idx = class_to_idx[cls_name]
    images = sorted([f for f in os.listdir(cls_path) if f.endswith((''.png', '.j

    for img_name in images:
        img_path = os.path.join(cls_path, img_name)
        image_paths.append(img_path)
        labels.append(cls_idx)

return image_paths, labels, class_to_idx, idx_to_class

def create_balanced_splits(image_paths, labels, train_size=0.7, val_size=0.15, test

X_train, X_temp, y_train, y_temp = train_test_split(
    image_paths, labels,
    test_size=(val_size + test_size),
    stratify=labels,
    random_state=SEED
)

val_ratio = val_size / (val_size + test_size)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp,
    test_size=(1 - val_ratio),
    stratify=y_temp,
    random_state=SEED
)

print(f"\nDataset splits:")
print(f"  Training: {len(X_train)} images")
print(f"  Validation: {len(X_val)} images")
print(f"  Test: {len(X_test)} images")

return X_train, X_val, X_test, y_train, y_val, y_test

def create_model_full_finetuning(num_classes, model_name='resnet50'):

    # Load pretrained ResNet
    if model_name == 'resnet18':
        model = models.resnet18(pretrained=True)
    elif model_name == 'resnet34':
        model = models.resnet34(pretrained=True)
    elif model_name == 'resnet50':
        model = models.resnet50(pretrained=True)
    elif model_name == 'resnet101':
        model = models.resnet101(pretrained=True)
    else:
        raise ValueError(f"Unsupported model: {model_name}")

```



```

# Step 1: Modify the final layer
num_fts = model.fc.in_features
model.fc = nn.Linear(num_fts, num_classes)

print(f"\n{'='*70}")
print(f"MODEL ARCHITECTURE: {model_name.upper()} (FULL FINE-TUNING)")
print(f"{'='*70}")
print(f"Original head input features: {num_fts}")
print(f"New head output classes: {num_classes}")

# Step 2: Make ALL layers trainable
for param in model.parameters():
    param.requires_grad = True

# Count parameters
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f"\nTotal parameters: {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
print(f"Frozen parameters: {total_params - trainable_params:,}")
print(f"Percentage trainable: {100 * trainable_params / total_params:.2f}%")

# Layer breakdown
print(f"\n{'Layer Trainability Status':^70}")
print("-" * 70)
for name, param in model.named_parameters():
    if 'fc' in name:
        print(f" {name:50s} | Trainable: {param.requires_grad}")
    elif 'layer4' in name and 'conv1' in name:
        print(f" {name:50s} | Trainable: {param.requires_grad}")
        break
print(f" ... (all {len(list(model.parameters()))} parameter tensors are trainable)")
print(f"{'='*70}\n")

return model

def train_one_epoch(model, dataloader, criterion, optimizer, device):

    model.train()
    running_loss = 0.0
    running_corrects = 0

    for images, labels in tqdm(dataloader, desc="Training", leave=False):
        images = images.to(device)
        labels = labels.to(device)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)
        loss = criterion(outputs, labels)
        _, preds = torch.max(outputs, 1)

        # Backward pass

```

```

        loss.backward()
        optimizer.step()

        # Statistics
        running_loss += loss.item() * images.size(0)
        running_corrects += torch.sum(preds == labels.data)

    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = running_corrects.double() / len(dataloader.dataset)

    return epoch_loss, epoch_acc.item()

def validate(model, dataloader, criterion, device):

    model.eval()
    running_loss = 0.0
    running_corrects = 0

    with torch.no_grad():
        for images, labels in tqdm(dataloader, desc="Validation", leave=False):
            images = images.to(device)
            labels = labels.to(device)

            # Forward pass
            outputs = model(images)
            loss = criterion(outputs, labels)
            _, preds = torch.max(outputs, 1)

            # Statistics
            running_loss += loss.item() * images.size(0)
            running_corrects += torch.sum(preds == labels.data)

    epoch_loss = running_loss / len(dataloader.dataset)
    epoch_acc = running_corrects.double() / len(dataloader.dataset)

    return epoch_loss, epoch_acc.item()

def train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
                num_epochs, device, save_path, patience=15):

    best_val_loss = float('inf')
    best_model_wts = copy.deepcopy(model.state_dict())
    epochs_no_improve = 0

    history = {
        'train_loss': [],
        'train_acc': [],
        'val_loss': [],
        'val_acc': [],
        'lr': [],
        'overfitting_gap': []
    }

    print(f"\n{'='*70}")
    print("TRAINING START - MONITORING FOR OVERFITTING")

```

```

print(f"{'='*70}\n")

for epoch in range(num_epochs):
    print(f"Epoch {epoch+1}/{num_epochs}")
    print("-" * 70)

    # Train
    train_loss, train_acc = train_one_epoch(model, train_loader, criterion, opt

    # Validate
    val_loss, val_acc = validate(model, val_loader, criterion, device)

    # Calculate overfitting gap
    overfitting_gap = train_acc - val_acc

    # Learning rate scheduling
    if scheduler is not None:
        scheduler.step()
        current_lr = scheduler.get_last_lr()[0]
    else:
        current_lr = optimizer.param_groups[0]['lr']

    # Store history
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)
    history['lr'].append(current_lr)
    history['overfitting_gap'].append(overfitting_gap)

    print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f}")
    print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")
    print(f"Overfitting Gap (Train-Val Acc): {overfitting_gap:.4f}")
    print(f"Learning Rate: {current_lr:.6f}")

    # Check for overfitting
    if overfitting_gap > 0.15:
        print(f" WARNING: Significant overfitting detected (gap: {overfitting_
    elif overfitting_gap > 0.10:
        print(f" CAUTION: Moderate overfitting (gap: {overfitting_gap:.2%})")

    # Save best model
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_model_wts = copy.deepcopy(model.state_dict())
        epochs_no_improve = 0
        torch.save({
            'epoch': epoch,
            'model_state_dict': best_model_wts,
            'optimizer_state_dict': optimizer.state_dict(),
            'val_loss': val_loss,
            'val_acc': val_acc,
        }, save_path)
        print(f" New best model saved! (Val Loss: {val_loss:.4f}, Val Acc: {val
    else:
        epochs_no_improve += 1

```

```

        print(f" No improvement for {epochs_no_improve} epoch(s)")

    print()

    # Early stopping
    if epochs_no_improve >= patience:
        print(f"Early stopping triggered after {epoch+1} epochs")
        break

    print(f"\n{'='*70}")
    print("TRAINING COMPLETE")
    print(f"{'='*70}")
    print(f"Best Validation Loss: {best_val_loss:.4f}")
    print(f"Model saved to: {save_path}\n")

    # Load best model weights
    model.load_state_dict(best_model_wts)

    return model, history

def plot_training_history(history, save_path):

    fig, axes = plt.subplots(2, 2, figsize=(16, 12))
    epochs = range(1, len(history['train_loss']) + 1)

    # Loss plot
    axes[0, 0].plot(epochs, history['train_loss'], 'b-', label='Training Loss', lin
    axes[0, 0].plot(epochs, history['val_loss'], 'r-', label='Validation Loss', lin
    axes[0, 0].set_xlabel('Epoch', fontsize=12, fontweight='bold')
    axes[0, 0].set_ylabel('Loss', fontsize=12, fontweight='bold')
    axes[0, 0].set_title('Training and Validation Loss', fontsize=14, fontweight='b
    axes[0, 0].legend(fontsize=11)
    axes[0, 0].grid(True, alpha=0.3)

    # Accuracy plot
    axes[0, 1].plot(epochs, history['train_acc'], 'b-', label='Training Accuracy',
    axes[0, 1].plot(epochs, history['val_acc'], 'r-', label='Validation Accuracy',
    axes[0, 1].set_xlabel('Epoch', fontsize=12, fontweight='bold')
    axes[0, 1].set_ylabel('Accuracy', fontsize=12, fontweight='bold')
    axes[0, 1].set_title('Training and Validation Accuracy', fontsize=14, fontweigh
    axes[0, 1].legend(fontsize=11)
    axes[0, 1].grid(True, alpha=0.3)

    # Overfitting gap plot
    axes[1, 0].plot(epochs, history['overfitting_gap'], 'g-', linewidth=2)
    axes[1, 0].axhline(y=0.10, color='orange', linestyle='--', label='Moderate Over
    axes[1, 0].axhline(y=0.15, color='red', linestyle='--', label='Significant Over
    axes[1, 0].set_xlabel('Epoch', fontsize=12, fontweight='bold')
    axes[1, 0].set_ylabel('Train Acc - Val Acc', fontsize=12, fontweight='bold')
    axes[1, 0].set_title('Overfitting Gap Analysis', fontsize=14, fontweight='bold'
    axes[1, 0].legend(fontsize=10)
    axes[1, 0].grid(True, alpha=0.3)

    # Learning rate plot
    axes[1, 1].plot(epochs, history['lr'], 'm-', linewidth=2)

```

```

axes[1, 1].set_xlabel('Epoch', fontsize=12, fontweight='bold')
axes[1, 1].set_ylabel('Learning Rate', fontsize=12, fontweight='bold')
axes[1, 1].set_title('Learning Rate Schedule', fontsize=14, fontweight='bold')
axes[1, 1].set_yscale('log')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.show()

print(f"Training curves saved to: {save_path}")

def evaluate_model(model, dataloader, device, class_names):

    model.eval()

    all_preds = []
    all_labels = []

    with torch.no_grad():
        for images, labels in tqdm(dataloader, desc="Evaluating"):
            images = images.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())

    all_preds = np.array(all_preds)
    all_labels = np.array(all_labels)

    return all_preds, all_labels

def plot_confusion_matrices(y_true, y_pred, class_names, accuracy, save_path):

    fig, axes = plt.subplots(1, 2, figsize=(24, 10))

    # Raw confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Purples', ax=axes[0],
                xticklabels=class_names, yticklabels=class_names,
                cbar_kws={'label': 'Count'})
    axes[0].set_title(f'Confusion Matrix - Raw Counts\n(Test Accuracy: {accuracy:.4}
                      fontsize=14, fontweight='bold')
    axes[0].set_ylabel('True Label', fontsize=12)
    axes[0].set_xlabel('Predicted Label', fontsize=12)
    plt.setp(axes[0].get_xticklabels(), rotation=45, ha='right', fontsize=9)
    plt.setp(axes[0].get_yticklabels(), rotation=0, fontsize=9)

    # Normalized confusion matrix
    cm_norm = confusion_matrix(y_true, y_pred, normalize='true')
    sns.heatmap(cm_norm, annot=True, fmt='.3f', cmap='Purples', ax=axes[1],
                xticklabels=class_names, yticklabels=class_names,
                cbar_kws={'label': 'Proportion'})
    axes[1].set_title('Normalized Confusion Matrix\n(Per-Class Recall)',

```

```

        fontsize=14, fontweight='bold')
axes[1].set_ylabel('True Label', fontsize=12)
axes[1].set_xlabel('Predicted Label', fontsize=12)
plt.setp(axes[1].get_xticklabels(), rotation=45, ha='right', fontsize=9)
plt.setp(axes[1].get_yticklabels(), rotation=0, fontsize=9)

plt.tight_layout()
plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.show()

print(f"Confusion matrices saved to: {save_path}")

# MAIN EXECUTION

print("="*70)
print("PROBLEM 2.c: FULL FINE-TUNING OF PRETRAINED MODEL")
print("="*70)

# Load dataset
print("\nLoading dataset...")
image_paths, labels, class_to_idx, idx_to_class = load_dataset(DATASET_PATH)
num_classes = len(class_to_idx)
class_names = [idx_to_class[i] for i in range(num_classes)]

# Create splits
X_train, X_val, X_test, y_train, y_val, y_test = create_balanced_splits(
    image_paths, labels, train_size=0.7, val_size=0.15, test_size=0.15
)

# Create datasets (with stronger augmentation for full fine-tuning)
train_dataset = UCMercedDataset(X_train, y_train, transform=train_transform)
val_dataset = UCMercedDataset(X_val, y_val, transform=test_transform)
test_dataset = UCMercedDataset(X_test, y_test, transform=test_transform)

# Create dataloaders
train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True,
    num_workers=NUM_WORKERS, prefetch_factor=PREFETCH_FACTOR, pin_memory=PIN_MEMORY
)
val_loader = DataLoader(
    val_dataset, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS, prefetch_factor=PREFETCH_FACTOR, pin_memory=PIN_MEMORY
)
test_loader = DataLoader(
    test_dataset, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=NUM_WORKERS, prefetch_factor=PREFETCH_FACTOR, pin_memory=PIN_MEMORY
)

# Create model with ALL Layers trainable
model = create_model_full_finetuning(num_classes, model_name='resnet50')
model = model.to(device)

# Step 2: Training Setup
criterion = nn.CrossEntropyLoss()

```

```

optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE) # ALL parameters
scheduler = optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.95)

print(f"{'='*70}")
print("TRAINING CONFIGURATION")
print(f"{'='*70}")
print(f"Learning Rate: {LEARNING_RATE} (LOWER for full fine-tuning)")
print(f"Optimizer: Adam (training ALL parameters)")
print(f"Loss Function: CrossEntropyLoss")
print(f"LR Scheduler: ExponentialLR (gamma=0.95)")
print(f"Batch Size: {BATCH_SIZE}")
print(f"Max Epochs: {NUM_EPOCHS}")
print(f"Early Stopping Patience: {PATIENCE}")
print(f>Data Augmentation: Enhanced (flips, rotation, color jitter)")
print(f"{'='*70}\n")

# Train model
model_save_path = os.path.join(SAVE_DIR, 'best_model_full_finetuning.pth')
model, history = train_model(
    model, train_loader, val_loader, criterion, optimizer, scheduler,
    NUM_EPOCHS, device, model_save_path, patience=PATIENCE
)

# Plot training history with overfitting analysis
plot_training_history(history, os.path.join(SAVE_DIR, 'training_curves_full_finetun

# STEP 3: EVALUATION AND COMPARISON

print("\n" + "="*70)
print("STEP 3: MODEL EVALUATION ON TEST SET")
print("="*70)

# Evaluate on test set
test_preds, test_labels = evaluate_model(model, test_loader, device, class_names)

# Calculate metrics
test_acc = accuracy_score(test_labels, test_preds)
precision, recall, f1, _ = precision_recall_fscore_support(
    test_labels, test_preds, average='weighted', zero_division=0
)

print(f"\n{'='*70}")
print("TEST SET PERFORMANCE (FULL FINE-TUNING)")
print(f"{'='*70}")
print(f"Accuracy: {test_acc:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"{'='*70}\n")

# Detailed classification report
print("Detailed Classification Report:")
print(classification_report(test_labels, test_preds, target_names=class_names))

```

```

# Plot confusion matrices
plot_confusion_matrices(
    test_labels, test_preds, class_names, test_acc,
    os.path.join(SAVE_DIR, 'confusion_matrices_full_finetuning.png')
)

# Per-class analysis
print("\n" + "="*70)
print("PER-CLASS PERFORMANCE ANALYSIS")
print("="*70)

class_precision, class_recall, class_f1, class_support = precision_recall_fscore_support(
    test_labels, test_preds, average=None, zero_division=0
)

f1_with_classes = list(zip(class_names, class_f1))
sorted_by_f1 = sorted(f1_with_classes, key=lambda x: x[1], reverse=True)

print("\n--- Top 5 Best Performing Classes ---")
for cls, f1_score in sorted_by_f1[:5]:
    idx = class_to_idx[cls]
    print(f"{cls:20s} - F1: {f1_score:.4f}, Precision: {class_precision[idx]:.4f},")

print("\n--- Top 5 Worst Performing Classes ---")
for cls, f1_score in sorted_by_f1[-5:]:
    idx = class_to_idx[cls]
    print(f"{cls:20s} - F1: {f1_score:.4f}, Precision: {class_precision[idx]:.4f},")

# Overfitting analysis
final_overfitting_gap = history['train_acc'][-1] - history['val_acc'][-1]
max_overfitting_gap = max(history['overfitting_gap'])

print("\n" + "="*70)
print("OVERFITTING ANALYSIS")
print("="*70)
print(f"Final Overfitting Gap: {final_overfitting_gap:.4f} ({final_overfitting_gap*100:.1f}%)")
print(f"Maximum Overfitting Gap: {max_overfitting_gap:.4f} ({max_overfitting_gap*100:.1f}%)")

if final_overfitting_gap < 0.05:
    print(" Excellent: Minimal overfitting")
elif final_overfitting_gap < 0.10:
    print(" Good: Acceptable overfitting level")
elif final_overfitting_gap < 0.15:
    print(" Moderate: Some overfitting present")
else:
    print(" High: Significant overfitting detected")

print("\n" + "="*70)
print("COMPARISON WITH PREVIOUS METHODS")
print("="*70)
print("\nTo compare with Problems 2.a and 2.b, note the following:")
print(" • Problem 2.a (Feature Extraction): Fixed features, separate classifier")
print(" • Problem 2.b (Head-Only Training): ~21K trainable parameters")
print(" • Problem 2.c (Full Fine-tuning): ~25M trainable parameters")
print("\nExpected outcomes:")
print(" • 2.c should achieve highest accuracy if overfitting is controlled")

```



```

print(" • 2.c requires longer training time")
print(" • 2.c adapts features specifically for aerial imagery")

print("\n" + "="*70)
print("EVALUATION COMPLETE")
print("="*70)
print(f"All results saved to: {SAVE_DIR}")
print(" - best_model_full_finetuning.pth")
print(" - training_curves_full_finetuning.png")
print(" - confusion_matrices_full_finetuning.png")
print("="*70)

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.
mount("/content/drive", force_remount=True).

Using device: cuda

```

=====
PROBLEM 2.c: FULL FINE-TUNING OF PRETRAINED MODEL
=====

```

Loading dataset...

Found 21 classes:

- 0: agricultural
- 1: airplane
- 2: baseballdiamond
- 3: beach
- 4: buildings
- 5: chaparral
- 6: denseresidential
- 7: forest
- 8: freeway
- 9: golfcourse
- 10: harbor
- 11: intersection
- 12: mediumresidential
- 13: mobilehomepark
- 14: overpass
- 15: parkinglot
- 16: river
- 17: runway
- 18: sparseresidential
- 19: storagetanks
- 20: tenniscourt

Dataset splits:

- Training: 1470 images
- Validation: 315 images
- Test: 315 images

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:627: UserWarning: This DataLoader will create 4 worker processes in total. Our suggested max number of worker in current system is 2, which is smaller than what this DataLoader is going to create. Please be aware that excessive worker creation might get DataLoader running slow or even freeze, lower the worker number to avoid potential slowness/freeze if necessary.
```

```
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
```

```
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet50_Weights.IMAGENET1K_V1`. You can also use `weights=ResNet50_Weights.DEFAULT` to get the most up-to-date weights.
```

```
warnings.warn(msg)
```

```
=====
MODEL ARCHITECTURE: RESNET50 (FULL FINE-TUNING)
=====
```

```
Original head input features: 2048
```

```
New head output classes: 21
```

```
Total parameters: 23,551,061
```

```
Trainable parameters: 23,551,061
```

```
Frozen parameters: 0
```

```
Percentage trainable: 100.00%
```

Layer Trainability Status

```
-----
layer4.0.conv1.weight | Trainable: True
... (all 161 parameter tensors are trainable)
=====
```

```
=====
TRAINING CONFIGURATION
```

```
Learning Rate: 0.0001 (LOWER for full fine-tuning)
```

```
Optimizer: Adam (training ALL parameters)
```

```
Loss Function: CrossEntropyLoss
```

```
LR Scheduler: ExponentialLR (gamma=0.95)
```

```
Batch Size: 32
```

```
Max Epochs: 100
```

```
Early Stopping Patience: 15
```

```
Data Augmentation: Enhanced (flips, rotation, color jitter)
```

```
=====
TRAINING START - MONITORING FOR OVERFITTING
=====
```

```
Epoch 1/100
-----
```

Train Loss: 1.3496 | Train Acc: 0.7007
Val Loss: 0.2648 | Val Acc: 0.9238
Overfitting Gap (Train-Val Acc): -0.2231
Learning Rate: 0.000095
New best model saved! (Val Loss: 0.2648, Val Acc: 0.9238)

Epoch 2/100

Train Loss: 0.2441 | Train Acc: 0.9395
Val Loss: 0.0919 | Val Acc: 0.9810
Overfitting Gap (Train-Val Acc): -0.0415
Learning Rate: 0.000090
New best model saved! (Val Loss: 0.0919, Val Acc: 0.9810)

Epoch 3/100

Train Loss: 0.1387 | Train Acc: 0.9639
Val Loss: 0.0959 | Val Acc: 0.9746
Overfitting Gap (Train-Val Acc): -0.0107
Learning Rate: 0.000086
No improvement for 1 epoch(s)

Epoch 4/100

Train Loss: 0.0932 | Train Acc: 0.9769
Val Loss: 0.0891 | Val Acc: 0.9714
Overfitting Gap (Train-Val Acc): 0.0054
Learning Rate: 0.000081
New best model saved! (Val Loss: 0.0891, Val Acc: 0.9714)

Epoch 5/100

Train Loss: 0.0920 | Train Acc: 0.9769
Val Loss: 0.0746 | Val Acc: 0.9778
Overfitting Gap (Train-Val Acc): -0.0009
Learning Rate: 0.000077
New best model saved! (Val Loss: 0.0746, Val Acc: 0.9778)

Epoch 6/100

Train Loss: 0.0836 | Train Acc: 0.9776
Val Loss: 0.0935 | Val Acc: 0.9714
Overfitting Gap (Train-Val Acc): 0.0061
Learning Rate: 0.000074
No improvement for 1 epoch(s)

Epoch 7/100

Train Loss: 0.0544 | Train Acc: 0.9891
Val Loss: 0.0741 | Val Acc: 0.9810
Overfitting Gap (Train-Val Acc): 0.0082
Learning Rate: 0.000070
New best model saved! (Val Loss: 0.0741, Val Acc: 0.9810)

Epoch 8/100

Train Loss: 0.0408 | Train Acc: 0.9905
Val Loss: 0.0563 | Val Acc: 0.9905
Overfitting Gap (Train-Val Acc): -0.0000
Learning Rate: 0.000066
New best model saved! (Val Loss: 0.0563, Val Acc: 0.9905)

Epoch 9/100

Train Loss: 0.0247 | Train Acc: 0.9939
Val Loss: 0.0473 | Val Acc: 0.9810
Overfitting Gap (Train-Val Acc): 0.0129
Learning Rate: 0.000063
New best model saved! (Val Loss: 0.0473, Val Acc: 0.9810)

Epoch 10/100

Train Loss: 0.0514 | Train Acc: 0.9864
Val Loss: 0.1308 | Val Acc: 0.9619
Overfitting Gap (Train-Val Acc): 0.0245
Learning Rate: 0.000060
No improvement for 1 epoch(s)

Epoch 11/100

Train Loss: 0.0455 | Train Acc: 0.9878
Val Loss: 0.0444 | Val Acc: 0.9873
Overfitting Gap (Train-Val Acc): 0.0005
Learning Rate: 0.000057
New best model saved! (Val Loss: 0.0444, Val Acc: 0.9873)

Epoch 12/100

Train Loss: 0.0259 | Train Acc: 0.9925
Val Loss: 0.0727 | Val Acc: 0.9746
Overfitting Gap (Train-Val Acc): 0.0179
Learning Rate: 0.000054
No improvement for 1 epoch(s)

Epoch 13/100

Train Loss: 0.0234 | Train Acc: 0.9939
Val Loss: 0.0595 | Val Acc: 0.9810
Overfitting Gap (Train-Val Acc): 0.0129
Learning Rate: 0.000051
No improvement for 2 epoch(s)

Epoch 14/100

Train Loss: 0.0174 | Train Acc: 0.9959
Val Loss: 0.0707 | Val Acc: 0.9841
Overfitting Gap (Train-Val Acc): 0.0118
Learning Rate: 0.000049
No improvement for 3 epoch(s)

Epoch 15/100

Train Loss: 0.0131 | Train Acc: 0.9973
Val Loss: 0.0771 | Val Acc: 0.9841
Overfitting Gap (Train-Val Acc): 0.0132
Learning Rate: 0.000046
No improvement for 4 epoch(s)

Epoch 16/100

Train Loss: 0.0175 | Train Acc: 0.9952
Val Loss: 0.0889 | Val Acc: 0.9841
Overfitting Gap (Train-Val Acc): 0.0111
Learning Rate: 0.000044
No improvement for 5 epoch(s)

Epoch 17/100

Train Loss: 0.0223 | Train Acc: 0.9932
Val Loss: 0.0467 | Val Acc: 0.9873
Overfitting Gap (Train-Val Acc): 0.0059
Learning Rate: 0.000042
No improvement for 6 epoch(s)

Epoch 18/100

Train Loss: 0.0115 | Train Acc: 0.9959
Val Loss: 0.0644 | Val Acc: 0.9841
Overfitting Gap (Train-Val Acc): 0.0118
Learning Rate: 0.000040
No improvement for 7 epoch(s)

Epoch 19/100

Train Loss: 0.0134 | Train Acc: 0.9973
Val Loss: 0.0534 | Val Acc: 0.9905
Overfitting Gap (Train-Val Acc): 0.0068
Learning Rate: 0.000038
No improvement for 8 epoch(s)

Epoch 20/100

Train Loss: 0.0106 | Train Acc: 0.9980
Val Loss: 0.0699 | Val Acc: 0.9905
Overfitting Gap (Train-Val Acc): 0.0075
Learning Rate: 0.000036
No improvement for 9 epoch(s)

Epoch 21/100

Train Loss: 0.0121 | Train Acc: 0.9980
Val Loss: 0.0617 | Val Acc: 0.9873
Overfitting Gap (Train-Val Acc): 0.0107
Learning Rate: 0.000034
No improvement for 10 epoch(s)

Epoch 22/100

Train Loss: 0.0098 | Train Acc: 0.9986
Val Loss: 0.0545 | Val Acc: 0.9905
Overfitting Gap (Train-Val Acc): 0.0082
Learning Rate: 0.000032
No improvement for 11 epoch(s)

Epoch 23/100

Train Loss: 0.0058 | Train Acc: 0.9993
Val Loss: 0.0541 | Val Acc: 0.9873
Overfitting Gap (Train-Val Acc): 0.0120
Learning Rate: 0.000031
No improvement for 12 epoch(s)

Epoch 24/100

Train Loss: 0.0067 | Train Acc: 0.9986
Val Loss: 0.0633 | Val Acc: 0.9873
Overfitting Gap (Train-Val Acc): 0.0113
Learning Rate: 0.000029
No improvement for 13 epoch(s)

Epoch 25/100

Train Loss: 0.0051 | Train Acc: 1.0000
Val Loss: 0.0560 | Val Acc: 0.9905
Overfitting Gap (Train-Val Acc): 0.0095
Learning Rate: 0.000028
No improvement for 14 epoch(s)

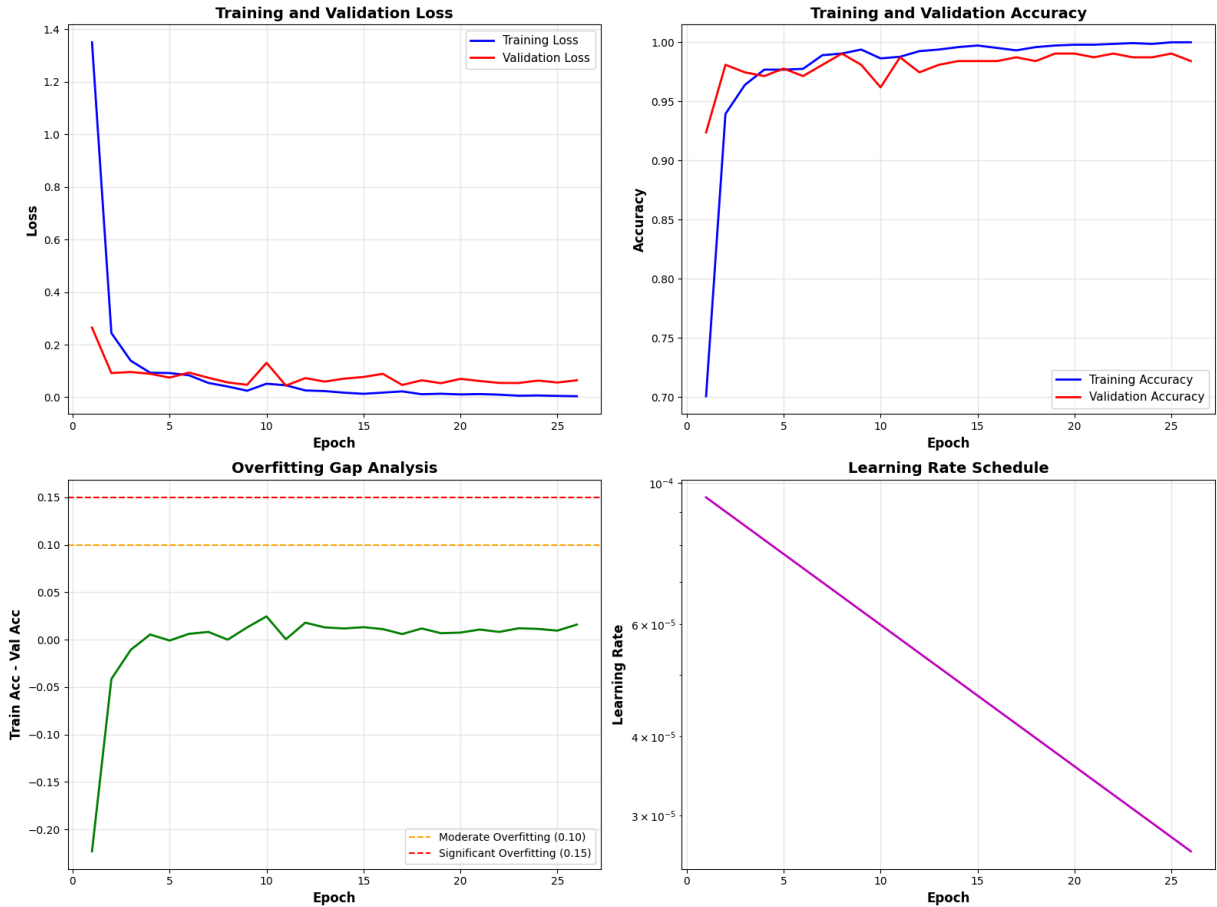
Epoch 26/100

Train Loss: 0.0038 | Train Acc: 1.0000
Val Loss: 0.0644 | Val Acc: 0.9841
Overfitting Gap (Train-Val Acc): 0.0159
Learning Rate: 0.000026
No improvement for 15 epoch(s)

Early stopping triggered after 26 epochs

TRAINING COMPLETE

Best Validation Loss: 0.0444
Model saved to: /content/drive/MyDrive/ucmerced_full_finetuning/best_model_full_fine
tuning.pth



Training curves saved to: /content/drive/MyDrive/ucmerced_full_finetuning/training_c
urves_full_finetuning.png

STEP 3: MODEL EVALUATION ON TEST SET

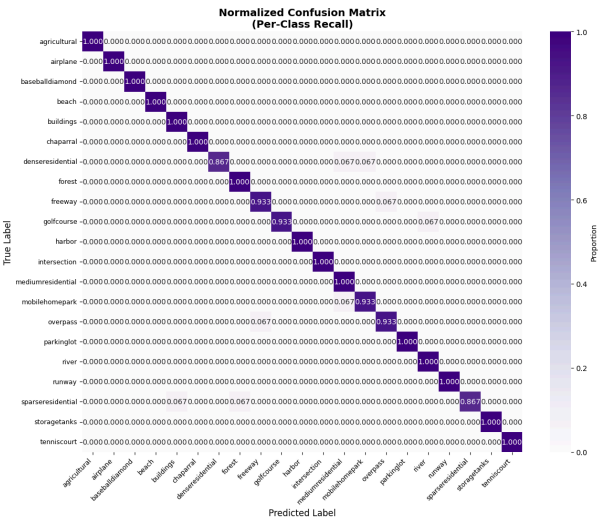
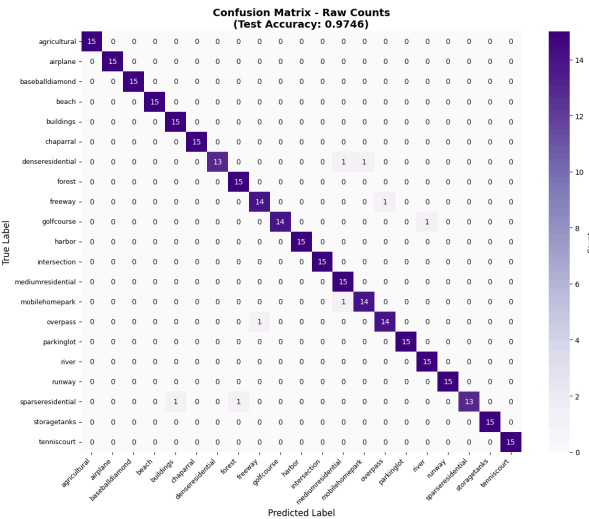
Evaluating: 100%|██████████| 10/10 [00:02<00:00, 4.49it/s]

TEST SET PERFORMANCE (FULL FINE-TUNING)

Accuracy: 0.9746
Precision: 0.9759
Recall: 0.9746
F1-Score: 0.9744

Detailed Classification Report:

	precision	recall	f1-score	support
agricultural	1.00	1.00	1.00	15
airplane	1.00	1.00	1.00	15
baseballdiamond	1.00	1.00	1.00	15
beach	1.00	1.00	1.00	15
buildings	0.94	1.00	0.97	15
chaparral	1.00	1.00	1.00	15
denseresidential	1.00	0.87	0.93	15
forest	0.94	1.00	0.97	15
freeway	0.93	0.93	0.93	15
golfcourse	1.00	0.93	0.97	15
harbor	1.00	1.00	1.00	15
intersection	1.00	1.00	1.00	15
mediumresidential	0.88	1.00	0.94	15
mobilehomepark	0.93	0.93	0.93	15
overpass	0.93	0.93	0.93	15
parkinglot	1.00	1.00	1.00	15
river	0.94	1.00	0.97	15
runway	1.00	1.00	1.00	15
sparseresidential	1.00	0.87	0.93	15
storagetanks	1.00	1.00	1.00	15
tenniscourt	1.00	1.00	1.00	15
accuracy			0.97	315
macro avg	0.98	0.97	0.97	315
weighted avg	0.98	0.97	0.97	315



Confusion matrices saved to: /content/drive/MyDrive/ucmerced_full_finetuning/confusion_matrices_full_finetuning.png

PER-CLASS PERFORMANCE ANALYSIS

--- Top 5 Best Performing Classes ---

agricultural	- F1: 1.0000, Precision: 1.0000, Recall: 1.0000
airplane	- F1: 1.0000, Precision: 1.0000, Recall: 1.0000
baseballdiamond	- F1: 1.0000, Precision: 1.0000, Recall: 1.0000
beach	- F1: 1.0000, Precision: 1.0000, Recall: 1.0000
chaparral	- F1: 1.0000, Precision: 1.0000, Recall: 1.0000

--- Top 5 Worst Performing Classes ---

freeway	- F1: 0.9333, Precision: 0.9333, Recall: 0.9333
mobilehomepark	- F1: 0.9333, Precision: 0.9333, Recall: 0.9333
overpass	- F1: 0.9333, Precision: 0.9333, Recall: 0.9333
denseresidential	- F1: 0.9286, Precision: 1.0000, Recall: 0.8667
sparseresidential	- F1: 0.9286, Precision: 1.0000, Recall: 0.8667

OVERFITTING ANALYSIS

Final Overfitting Gap: 0.0159 (1.59%)
Maximum Overfitting Gap: 0.0245 (2.45%)
Excellent: Minimal overfitting

COMPARISON WITH PREVIOUS METHODS

To compare with Problems 2.a and 2.b, note the following:

- Problem 2.a (Feature Extraction): Fixed features, separate classifier
- Problem 2.b (Head-Only Training): ~21K trainable parameters
- Problem 2.c (Full Fine-tuning): ~25M trainable parameters

Expected outcomes:

- 2.c should achieve highest accuracy if overfitting is controlled
- 2.c requires longer training time
- 2.c adapts features specifically for aerial imagery

EVALUATION COMPLETE

All results saved to: /content/drive/MyDrive/ucmerced_full_finetuning

- best_model_full_finetuning.pth
- training_curves_full_finetuning.png
- confusion_matrices_full_finetuning.png

Comparison of Three Approaches

The three methods demonstrated a clear progression in performance: feature extraction with SVM achieved 96.83% accuracy, head-only fine-tuning reached 94.60%, and full fine-tuning

achieved the highest accuracy at 97.46%. The full fine-tuning approach outperformed both previous methods by allowing the entire network to adapt its learned ImageNet representations specifically to the remote sensing domain, resulting in more discriminative features tailored to the UCMerced dataset's unique characteristics. Interestingly, the head-only fine-tuning underperformed the SVM approach, likely because freezing the backbone prevented the model from adapting mid-level features (textures, patterns) that differ significantly between natural ImageNet scenes and aerial imagery, while the single trainable layer lacked the optimization power of SVM's margin maximization. The full fine-tuning's success validates that despite the small dataset size (2,100 images total), careful regularization through lower learning rates (10^{-4} vs 10^{-3}) and leveraging pre-trained weights as initialization effectively mitigated overfitting while enabling domain-specific adaptation.

Performance Improvement Strategies

Several avenues exist for further improvement. Model architecture: Experimenting with larger ResNet variants (ResNet101, ResNet152) or modern architectures like EfficientNet, Vision Transformers (ViT), or ConvNeXt could capture more complex spatial relationships in aerial imagery; additionally, models pre-trained specifically on satellite/aerial imagery datasets (like those from remote sensing competitions) would provide more relevant initial features than ImageNet weights. Training enhancements: Implementing data augmentation techniques such as random rotations, flips, color jittering, and cutout/mixup would artificially expand the training set and improve robustness; using techniques like progressive unfreezing (gradually unfreezing layers from head to backbone) or discriminative learning rates (lower rates for early layers, higher for later layers) could provide more controlled fine-tuning. Generalization improvements: Cross-validation with multiple train-test splits would provide more robust performance estimates; ensemble methods combining predictions from multiple fine-tuned models or architectures could reduce variance; and incorporating label smoothing or focal loss could address any class imbalance issues and improve calibration.

In [3]: